A Sling Against Giants

LightPFN, a 4.6M-parameter tabular in-context classifier designed to stay small

Giorgio Ottoboni

Technical report, version 1.2, 28 October 2026

https://github.com/GioOtto/LightPFN

Abstract

We present LightPFN, a tabular in-context classifier with 4.60 million parameters, pretrained exclusively on synthetic tasks without distillation. Its three-stage architecture combines summary-mediated row refinement with a target-aware column stage; pretraining mixes structural causal graph and rule priors. Model selection uses held-out synthetic tasks, mechanism probes and real datasets outside TabArena. The final model achieves a mean AUC of 0.911 on 55 external small-table datasets, 0.9 percentage points above default CatBoost. A long-context training stage improves performance on larger tables: mean AUC differences from default CatBoost have 95% bootstrap intervals including zero at 4,000–32,000 training rows and, in a separate evaluation on larger datasets, at 10,000–100,000 rows. On 38 TabArena classification tasks, using the official splits of one repeat, four estimators rank second among seven evaluated configurations, behind TabICLv2, and yield lower error than default CatBoost on 76% of tasks. On the official TabArena leaderboard as of 11 October 2026, where the maintainers ran and verified it on all splits of the 38 classification datasets, LightPFN ranks 21st of 92 methods with Elo 1441 (+69/−79). Its point estimate exceeds every tree-based entry, including tuned and ensembled ones, but its interval overlaps those of default CatBoost and of the tuned gradient-boosting entries. Cache blocking and folded normalization reduce CPU inference time by 1.1–4.1 times with predictions unchanged up to floating-point rounding. Future work will assess inference latency and memory use on less powerful computers, including laptops.

Keywords: tabular foundation models · in-context learning · synthetic priors · small models · CPU inference

1 Introduction

Gradient-boosted decision trees remain the default choice for tabular classification. They are widely used on CPUs and remain strong baselines after tuning [7]. Prior-data fitted networks (PFNs) [16, 9] approach the problem from a different angle: a transformer is trained once on millions of synthetic datasets drawn from a prior over data-generating processes, and at test time it reads a labelled training set as context and predicts the test labels in a single forward pass. It learns a predictive rule for new datasets, so the synthetic problems used for training and the capacity of the network determine what it can learn. TabPFN v2 [10], TabICL [21] and their successors have shown that this recipe matches or exceeds tuned boosting on public benchmarks such as TabArena [7], with models ranging from a few million to several hundred million parameters, usually served on GPU.

This report describes LightPFN, for which we impose three design constraints. The parameter budget stays below five million, with the goal of keeping inference practical on a commodity CPU; a small parameter count does not by itself ensure prediction times comparable to boosted trees. Training uses synthetic data only, with no distillation and no weights or predictions of other tabular foundation models. Code, weights and the

training pipeline are released under the Apache 2.0 license (https://github.com/GioOtto/LightPFN, package `LightPFN` on PyPI); dependency licenses still apply. Finally, after introducing the selection protocol, design choices use sets outside TabArena, which is reserved for evaluating the finalists.

The practical target is the way boosted trees are used day to day: with default settings, on a laptop. A model that matches or beats default CatBoost [19], LightGBM [12] and XGBoost [4] without tuning and without a GPU is useful even where tuned and ensembled boosting stays ahead, so we report the comparison with default tree ensembles alongside the selection sets.

Our contributions are the following.

1. An adaptation and evaluation of refinement before compression [5] under a sub-five-million parameter budget. The model learns interactions between features before compressing each row into a fixed-size vector, improving accuracy over our baseline without increasing its size.
2. A rule prior that includes tasks whose labels depend on combinations of features while remaining independent of each individual feature. Mixed at 10% with a graph prior, it teaches interactions that the graph prior alone does not convey, provided the row stage has the capacity to combine features.
3. A selection protocol based on paired bootstrap comparisons with false discovery rate control over synthetic, probe and real datasets, together with a hardware anchor that measures how much a single-seed difference can move without any change in the recipe.
4. A reproducible training and inference system, including fresh synthetic data generation, reduced CPU time and memory use, and a Vulkan backend for GPUs from different vendors. Evaluations on large tables, categorical features and TabArena establish both the gains and the remaining limitations.

Section 2 introduces PFNs and the statistical and architectural concepts used below. Sections 3–5 describe the model, synthetic generators and training procedure. Sections 6–9 present the selection protocol, ablations, comparisons with default tree ensembles and large-table experiments; the earlier exploratory TabArena evaluation is disclosed in Section 7.6. Sections 10–12 cover inference (including Vulkan), the final TabArena evaluation and categorical features. The report closes with limitations and conclusions.

2 Background

Prior-data fitted networks. A PFN learns a prediction procedure across simulated datasets rather than fitting a separate set of network weights to each new dataset. A synthetic generator defines which prediction problems it encounters during pretraining; this distribution over problems is the *prior*. At deployment, labelled rows supply evidence about the new problem, while the pretrained network weights remain fixed [16].

For one dataset, write $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$. There are n rows, m features and C classes; $x_i \in (\mathbb{R} \cup \{\varnothing\})^m$ is the feature vector of row i, with $\varnothing$ marking a missing value, and $y_i \in \{1, \dots, C\}$ is its class label. The *context* $\mathcal{D}_{\text{ctx}}$ contains n_c labelled rows. The remaining n_q rows, indexed by Q, are *queries*: their features are supplied to the network, but their labels are withheld and used to score its predictions. Thus $n = n_c + n_q$. A task is one such context/query prediction problem. Let $p(\mathcal{D})$ denote the distribution over datasets induced by the generator. The PFN $q_\theta(y \mid x, \mathcal{D}_{\text{ctx}})$ returns class probabilities, with θ denoting its learned network parameters. Pretraining minimizes the expected negative log probability of the withheld labels:

$$\mathcal{L}(\theta) = \mathbb{E}_{\mathcal{D}\sim p(\mathcal{D})}\Big[-\frac{1}{n_q} \sum_{i\in Q} \log q_\theta\big(y_i \mid x_i, \mathcal{D}_{\text{ctx}}\big)\Big]. \tag{1}$$

The expectation includes the context/query split. Over unrestricted conditional predictors, the population minimizer is $p(y \mid x, \mathcal{D}_{\text{ctx}})$ under this task distribution [16, 17]. To give its Bayesian interpretation, let $\phi \sim p(\phi)$ describe a data-generating mechanism (for example its graph, functions and noise parameters),

distinct from the network weights θ. If rows are conditionally independent and identically distributed given ϕ, with a context/query split independent of their values, then

$$p(y \mid x, \mathcal{D}_{\text{ctx}}) = \int p(y \mid x, \phi)\, p(\phi \mid x, \mathcal{D}_{\text{ctx}})\, d\phi. \tag{2}$$

This *posterior predictive distribution* averages predictions over mechanisms compatible with the observed context and query features. The familiar form with $p(\phi \mid \mathcal{D}_{\text{ctx}})$ requires that observing x supply no further information about the conditional response model, as when the prior over that model is independent of the feature distribution [17]. We do not assume this factorization for the graph generators. A finite trained PFN approximates the predictive rule induced by its actual generator, including filtering and preprocessing; it does not explicitly compute this integral, nor is exact Bayesian inference guaranteed on real datasets.

Three-stage architectures. TabICL [21], its successor TabICLv2 [22] and TabPFN-3 [20] process a table in three stages: an embedding of every cell refined by attention along each column, a transformer along each row that compresses the row into a fixed-size vector, and an in-context learning (ICL) transformer over the row vectors in which test rows attend to training rows. With fixed depth and width, induced column attention costs $O(nmk)$ for k inducing tokens, while full within-row self-attention costs $O(nm^2)$. The ICL attention costs $O(n_c^2 + n_q n_c)$ and operates on fixed-width row embeddings, independently of the feature count. Causilo [5] introduces row refinement, a second column stage and a separate row readout through fixed-size summaries. LightPFN adapts this pattern: its row attention is linear in m, while the quadratic context term in ICL remains.

Priors and structural causal models. TabPFN and TabICL draw tasks from structural causal models (SCMs) [9, 21, 22]. An SCM specifies variables as functions of their parent variables and noise on a directed acyclic graph (DAG), a directed graph with no directed cycles. To simulate a table, one samples a graph and node functions, then generates rows by evaluating the nodes in an order that respects the arrows. Selected nodes supply the observed features and target. Resampling the graph and functions between tasks induces a distribution over prediction problems. Here these graphs generate synthetic dependencies; they are not graphs inferred from a real dataset. Section 4 distinguishes this inherited graph generator from our rule-based labelling procedure. Mitra [27] showed that adding tasks labelled by fitted tree ensembles to the mixture improves real-data performance. LimiX [26] trains on masked joint modelling, predicting held-out cells as well as labels. Our work uses ideas from these papers and none of their weights.

3 Model

We describe the architecture of LightPFN selected for the final run, denoted B4 in the ablations. A0 denotes our baseline: one column stage, full within-row self-attention without refinement, and eight ICL blocks. For one task, n_c labelled context rows and n_q unlabelled query rows share m features, with $n = n_c + n_q$. Cell states have width $d_e = 64$ until row compression; row states then have width $d_r = 256$.

Figure 1 provides a reading guide. Panel 1 is the input table X, with labels available only for the context. Panels 2–5 replace each scalar cell with a vector, compare it with the labelled cells in its column, exchange information between features in each row, and revisit the columns. Panel 6 compresses each row into one vector. Panel 7 compares query rows with the labelled context, and panel 8 turns these comparisons into class probabilities. The corresponding operators are Φ, $\mathcal{C}_1$, $\mathcal{F}$, $\mathcal{C}_2$, $\mathcal{P}$, $\mathcal{I}$ and $\mathcal{D}$, respectively; $P \in \mathbb{R}^{n_q \times C}$ collects the predicted probabilities:

$$X \xrightarrow{\Phi} E^{(0)} \xrightarrow{\mathcal{C}_1} E^{(1)} \xrightarrow{\mathcal{F}} \bar{E} \xrightarrow{\mathcal{C}_2} E^{\star} \xrightarrow{\mathcal{P}} R \xrightarrow{\mathcal{I}} (Z, U) \xrightarrow{\mathcal{D}} P. \tag{3}$$

The two column stages use independent parameters. The refinement retains individual cells; only the pooling operator $\mathcal{P}$ removes the feature axis. Context statistics and shared attention memories depend on labelled rows alone, so query rows can be processed independently. Figures 1 and 2 show this information flow and the attention operators.

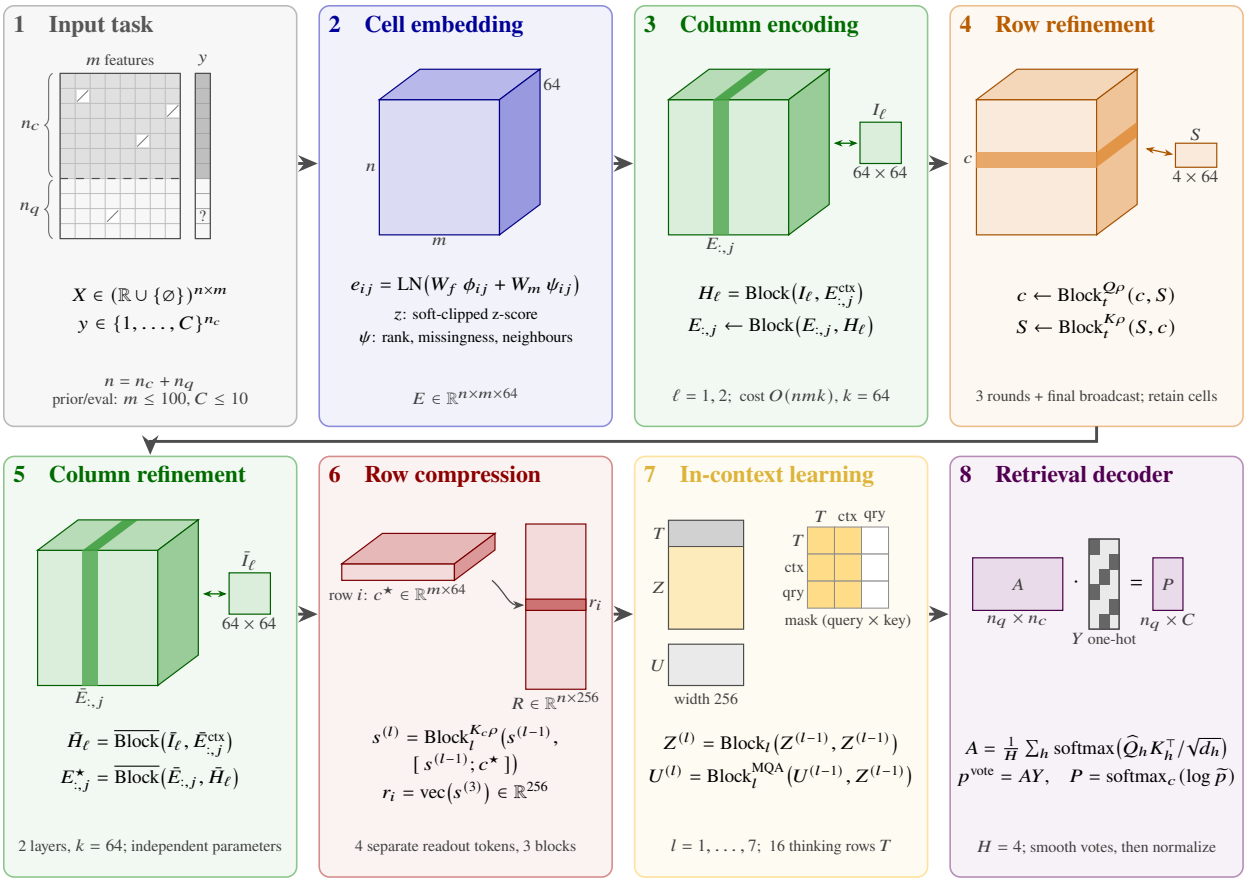


Figure 1. B4 from the input table through seven operators. Both column stages gather from labelled context cells and broadcast to context and query cells. Row refinement exchanges information through temporary summaries while retaining the cells; row compression uses a separate set of summaries. The second column stage is an explicit step between refinement and compression. Tensor shapes, attention directions and the context/query mask are shown for one task.

3.1 Attention block

Attention lets each input vector retrieve a weighted average of other vectors. A *query* vector is compared with source *keys*; a softmax turns their similarity scores into positive weights summing to one, which are applied to source *values*. These attention queries are internal vectors, distinct from the unlabelled query rows of a task. Multiple heads learn separate comparisons and concatenate their outputs. A token is one vector in such a sequence; learned summary tokens provide a fixed number of vectors through which a variable-sized set can exchange information. Figure 2 shows how these operations are used in LightPFN.

Attention modules use the same pre-norm residual block structure, with their own parameters: inputs are normalized before each transformation, and its output is added to the input.

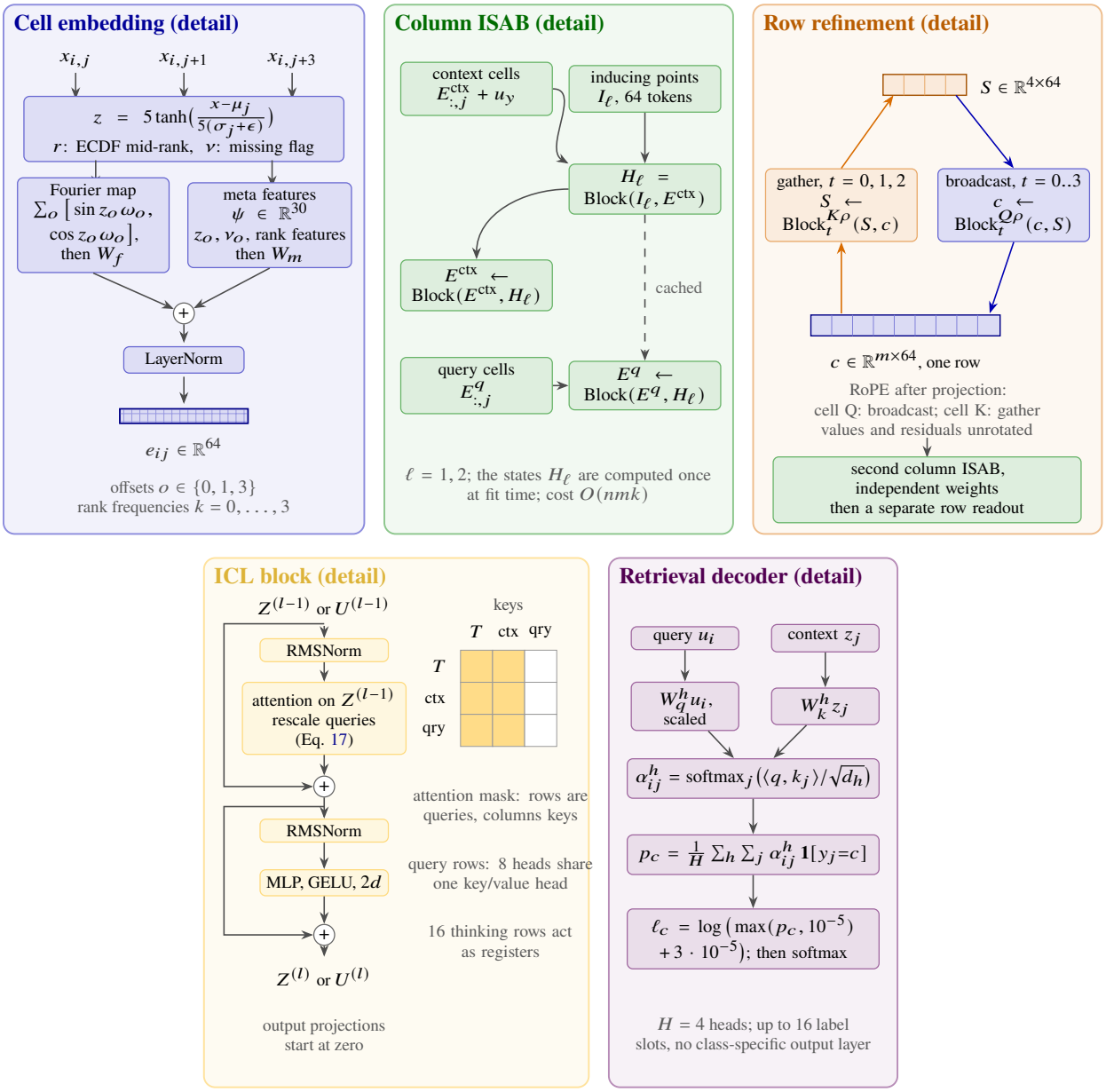


Figure 2. Cell embedding, column ISAB, row refinement, an ICL block and label retrieval. The dashed column arrow denotes cached memory. Rotary positions act on projected cell queries in broadcast and on projected cell keys in gather and pooling. Cell values and residual states are unrotated. The column pattern is used twice with independent parameters.

Let $X \in \mathbb{R}^{L_q \times d}$ be the query sequence and $Y \in \mathbb{R}^{L_k \times d}$ the source sequence. Here X, Y are generic block inputs, not the raw table and labels; L_q, L_k are sequence lengths, d is the vector width and d_h the width of head h. Each head projects separately normalized states into queries, keys and values, using the output-by-input weight convention of the implementation:

$$\begin{aligned} Q_h &= \mathrm{RMSNorm}_Q(X) W_{Q,h}^\top, \\ K_h &= \mathrm{RMSNorm}_{KV}(Y) W_{K,h}^\top, \qquad V_h = \mathrm{RMSNorm}_{KV}(Y) W_{V,h}^\top, \end{aligned} \tag{4}$$

$$\widetilde{X} = X + \mathrm{Concat}_h \left[\mathrm{softmax}\left(\frac{\widehat{Q}_h \widehat{K}_h^\top}{\sqrt{d_h}} + M \right) V_h \right] W_O^\top, \tag{5}$$

$$\mathrm{Block}(X, Y) = \widetilde{X} + \mathrm{GELU}\big(\mathrm{RMSNorm}_{FF}(\widetilde{X}) W_1^\top \big) W_2^\top. \tag{6}$$

Here RMSNorm normalizes by the root mean square of a vector with a learned scale, GELU is a smooth nonlinear activation, and the W matrices are learned linear projections. M is an additive mask with zero on allowed keys and $-\infty$ on excluded keys, and $W_1 \in \mathbb{R}^{2d \times d}$. Ordinarily $\widehat{Q}_h = Q_h$ and $\widehat{K}_h = K_h$; positional rotation or query scaling modifies these projections where specified below. We write $\mathrm{Block}^{Q\rho}$ when rotary positions act on cell queries, $\mathrm{Block}^{K\rho}$ when they act on cell keys, and $\mathrm{Block}^{K_c\rho}$ when only the cell subset of a mixed source is rotated. The values and residual state are unchanged by these positional operations. Output projections W_O and W_2 start at zero, so each block is the identity at initialization. Cached source keys and values are projected once and reused for subsequent queries.

3.2 Cell embedding

The cell embedding (panel 2 of Figure 1, top left of Figure 2) converts each scalar entry x_{ij}, row i and column j, into a 64-dimensional vector. Statistics are computed on the context rows of each column j, ignoring missing values: the mean μ_j, the standard deviation σ_j and the sorted values. A cell is described by a soft-clipped z-score, a mid-rank under the empirical cumulative distribution function (CDF) of the context, and a missingness flag. Let I_j index the observed context values of column j and $N_j = |I_j|$. For $N_j > 0$ and an observed cell,

$$\begin{aligned} z_{ij} &= 5 \tanh\!\left(\frac{x_{ij} - \mu_j}{5(\sigma_j + \epsilon)} \right), \qquad \nu_{ij} = \mathbf{1}[x_{ij} \text{ missing}], \\ r_{ij} &= \frac{\#\{\ell \in I_j : x_{\ell j} < x_{ij}\} + \#\{\ell \in I_j : x_{\ell j} \le x_{ij}\}}{2\, N_j}. \end{aligned} \tag{7}$$

Here # counts set elements, $\mathbf{1}$ is an indicator and $\epsilon > 0$ stabilizes division; missing cells get $z_{ij} = 0$, $r_{ij} = 1/2$ and $\nu_{ij} = 1$. The standard deviation uses denominator $\max(N_j - 1, 1)$; constant columns receive $z = 0$, and entirely unobserved columns receive $r = 1/2$. The mean and standard deviation are conventional sample statistics: soft clipping bounds the normalized values but does not make these estimators robust. Each cell also sees two neighbouring columns through circular offsets $o \in \{0, 1, 3\}$, which gives the embedding a cheap view of local feature pairs. With learned frequencies $\omega_o \in \mathbb{R}^{16}$, the value part is a Fourier feature map [25], and the rank part uses fixed dyadic frequencies:

$$e_{ij}^{\mathrm{val}} = W_f \sum_o \big[\sin(z_{i,j+o}\, \omega_o),\ \cos(z_{i,j+o}\, \omega_o) \big], \tag{8}$$

$$e_{ij}^{\mathrm{meta}} = W_m \psi_{ij}, \qquad \psi_{ij} = \big[(z_{i,j+o})_o;\ (\nu_{i,j+o})_o;\ (\sin(2^k \pi r_{i,j+o}))_{o,k};\ (\cos(2^k \pi r_{i,j+o}))_{o,k} \big] \in \mathbb{R}^{30}, \tag{9}$$

$$e_{ij} = \mathrm{LN}\left(e_{ij}^{\mathrm{val}} + e_{ij}^{\mathrm{meta}} \right) \in \mathbb{R}^{64}. \tag{10}$$

Here LN is layer normalization, W_f, W_m are learned projections, $o \in \{0, 1, 3\}$ and $k = 0, \ldots, 3$; parentheses with subscript o or (o, k) collect the corresponding entries into a vector, and semicolons denote concatenation. Offsets wrap modulo the true feature count; padding is excluded from feature attention. All statistics are fitted

to the context and reused for queries. Categorical columns enter as ordinal codes of their categories, through the same embedding as numeric columns; the network has no categorical-specific encoding (Section 12).

3.3 Column encoding

The first column operator C_1 uses induced set attention blocks (ISAB) [13]. In panel 3 of Figure 1 and the top-centre panel of Figure 2, *gather* summarizes labelled cells into a fixed set of inducing tokens, and *broadcast* lets each context or query cell read those summaries. It acts independently on the row sequence of each column, with parameters shared across columns. Context cells start from $E^{(0)}_{ij} + u^{(1)}_{y_i}$; query cells start from $E^{(0)}_{ij}$. Label embeddings have orthonormal initialization and are subsequently learned without an orthogonality constraint. With $k = 64$ learned inducing tokens I_ℓ per layer, the updates are

$$H_{\ell j} = \mathrm{Block}^{\mathrm{ind}}_\ell(I_\ell, E^{\mathrm{ctx}}_{:,j}), \qquad E^a_{:,j} \leftarrow \mathrm{Block}^{\mathrm{cell}}_\ell(E^a_{:,j}, H_{\ell j}), \quad a \in \{\mathrm{ctx}, q\}, \quad \ell = 1, 2. \tag{11}$$

The gather and broadcast blocks have separate parameters; the broadcast weights are shared between context and query cells. Query cells never contribute to $H_{\ell j}$. For prediction, the projected keys and values of each $H_{\ell j}$ are cached. This makes the encoding sensitive to the labelled column distribution, with attention cost $O(nmk)$ at fixed depth and width.

3.4 Row refinement

The operator $\mathcal{F}$ exchanges information across the feature axis while retaining all cell states (panel 4 of Figure 1, top right of Figure 2). It follows the summary-mediated refinement of Causilo [5]. Each row starts with its cells $c^{(0)} \in \mathbb{R}^{m\times 64}$ and $K = 4$ learned temporary tokens $S^{(0)} \in \mathbb{R}^{K\times 64}$, initialized identically across rows. For $t = 0, \ldots, R-1$,

$$c^{(t+1)} = \mathrm{Block}^{\mathrm{bc},Q\rho}_t\left(c^{(t)}, S^{(t)}\right), \tag{12}$$

$$S^{(t+1)} = \mathrm{Block}^{\mathrm{ga},K\rho}_t\left(S^{(t)}, c^{(t+1)}\right), \tag{13}$$

followed by a final broadcast

$$\bar{c} = \mathrm{Block}^{\mathrm{bc},Q\rho}_R\left(c^{(R)}, S^{(R)}\right). \tag{14}$$

With $R = 3$, there are four broadcast and three gather blocks, all independently parameterized. Rotary position embedding [24] acts on projected cell queries during broadcast and projected cell keys during gather. Only after the first gather do the temporary tokens carry row-specific evidence from other features. The final broadcast returns this evidence to the cells; the temporary tokens are then discarded. The attention cost is $O((2R+1)nmK)$, linear in m for fixed R and K, compared with $O(nm^2)$ for the baseline's full row self-attention.

3.5 Column refinement

The operator C_2 applies the same two-layer induced attention pattern as C_1 (panel 5 of Figure 1), with independent inducing tokens, projections, feed-forward blocks and label embeddings $u^{(2)}_y$. Its context input is $\bar{E}^{\mathrm{ctx}}_{ij} + u^{(2)}_{y_i}$ and its query input is $\bar{E}^q_{ij}$. The context-only gather now reads cells that already encode within-row feature interactions. The broadcast compares each such cell with this enriched column memory before any feature information is pooled away. The second stage has its own cached keys and values, and the same $O(nmk)$ attention cost as the first.

3.6 Row compression

The pooling operator $\mathcal{P}$ (panel 6 of Figure 1) uses $K_p = 4$ learned readout tokens $s^{(0)} \in \mathbb{R}^{K_p\times 64}$, separate from the temporary tokens of $\mathcal{F}$. For each row, three attention blocks update these readout tokens while

holding the refined cells $c^\star$ fixed:

$$s^{(l)} = \mathrm{Block}_l^{K_c\rho}\left(s^{(l-1)}, [\, s^{(l-1)}; c^\star \,]\right), \qquad r_i = \mathrm{vec}\big(s^{(3)}\big) \in \mathbb{R}^{256}. \tag{15}$$

Only the cell keys receive rotary positions; summary keys and all values remain unrotated. The attention cost is $O(nK_p(m + K_p))$ at fixed depth. This operation removes the feature axis and supplies a fixed-width representation to ICL. B3, a pooling-only variant of A0, ablated this pooling in place of full row self-attention: its accuracy differences from A0 were inconclusive, and its training time was 70% of A0's. This supports it as an efficient readout, without establishing statistical equivalence.

3.7 In-context learning stage

At this stage each row is one vector (panel 7 of Figure 1, bottom left of Figure 2); attention now compares rows rather than individual cells. Context rows receive a row-level label embedding $v_{y_i} \in \mathbb{R}^{256}$ and are preceded by 16 learned thinking rows T, which act as registers [6]. With $Z^{(0)} = [\, T;\ R_{\mathrm{ctx}} + V_y \,]$ and query rows $U^{(0)} = R_q$, each of the $L = 7$ blocks computes

$$Z^{(l)} = \mathrm{Block}_l\left(Z^{(l-1)}, Z^{(l-1)}\right), \qquad U^{(l)} = \mathrm{Block}_l^{\mathrm{MQA}}\left(U^{(l-1)}, Z^{(l-1)}\right). \tag{16}$$

Context and query updates share block parameters, but use different key/value head patterns. Context rows use eight-head self-attention; query rows attend to the context and thinking rows using multi-query attention [23], with all eight query heads reading the first key/value head. Query rows never attend to each other, and the context never reads them. The prediction cache therefore stores one eighth of the full per-layer ICL keys and values; this cache ratio does not imply an eightfold reduction in total inference cost. To adapt attention sharpness to the number of source keys N, projected queries are rescaled by

$$q \leftarrow q \odot \beta(\log \max(N, 2)) \odot \big(1 + \tanh \varphi(q)\big), \tag{17}$$

where β and φ are learned MLPs initialized to make scaling the identity, and $N = n_c + 16$ in ICL. Here q is a projected attention-query vector and $\odot$ denotes elementwise multiplication; MLP means multilayer perceptron. A final RMS normalization is applied to context and query row states.

3.8 Retrieval decoder

The readout is an attention-weighted vote of the context labels (panel 8 of Figure 1, bottom right of Figure 2). With $H = 4$ heads, queries from the final query states u_i and keys from the final context states z_j,

$$\begin{aligned} \alpha_{ij}^h &= \mathrm{softmax}_j\left(\frac{\langle \widehat{q}_i^h,\ W_k^h z_j \rangle}{\sqrt{d_h}}\right), \\ p_{ic}^{\mathrm{vote}} &= \frac{1}{H}\sum_{h=1}^{H}\sum_{j \in \mathcal{D}_{\mathrm{ctx}}} \alpha_{ij}^h\, \mathbf{1}[y_j = c]. \end{aligned} \tag{18}$$

Here $\langle a, b \rangle = a^\top b$ is the Euclidean inner product, c indexes a class, and $\widehat{q}_i^h$ is $W_q^h u_i$ after key-count scaling with $N = n_c$; the decoder has its own scaling parameters. One-hot context labels supply the values, and the thinking rows are excluded. The decoder returns logits

$$\begin{aligned} \widetilde{p}_{ic} &= \max(p_{ic}^{\mathrm{vote}}, 10^{-5}) + 3 \cdot 10^{-5}, \qquad \ell_{ic} = \log \widetilde{p}_{ic}, \\ q_\theta(y = c \mid x_i, \mathcal{D}_{\mathrm{ctx}}) &= \mathrm{softmax}_c(\ell_i)_c. \end{aligned} \tag{19}$$

The smoothing bounds the loss of a confident mistake; the final softmax renormalizes the smoothed votes across classes. In $\mathrm{softmax}_c(\ell_i)_c$, the subscript on the operator denotes normalization over classes and the final subscript selects class c. No learned output vector is needed per class. Although the attention value width is 64, the complete model's label-embedding interface supports at most 16 classes; the training prior contains 2–10 classes, so support for 11–16 is a capacity limit rather than an evaluated guarantee.

Component	A0	B4
Cell embedding	4,144	4,144
Column encoding	141,056	141,056
Row refinement ($R = 3$)	0	230,976
Column refinement	0	141,056
Row encoder / compression	99,136	99,136
ICL, including labels, registers and final norm	4,376,128	3,830,560
Retrieval decoder	156,160	156,160
Total trainable parameters	4,777,072	4,603,088

Table 1. Trainable parameter counts. Each ICL block contains 546,016 parameters. The 372,032 parameters of row and column refinement are funded by reducing ICL depth from eight to seven. The baseline row encoder and B4 pooling have the same parameter count but different attention patterns. B4 is 173,984 parameters smaller than A0.

Label slots. The model has 16 label slots. During training the C classes of every task are mapped to a random injective assignment of slots, exposing all slot embeddings to training and discouraging dependence on a fixed class index. Random slot assignment does not impose exact class-permutation equivariance on the learned embeddings.

3.9 Symmetries and attention cost

In exact arithmetic, permuting context rows together with their labels leaves query predictions unchanged: column summaries and ICL have no row-position encoding, and retrieval sums over the permuted context. Permuting query rows permutes their outputs, and a query's prediction does not depend on other queries. Finite-precision reductions can introduce small numerical differences. Feature-order invariance is not imposed: both circular feature grouping and rotary positions depend on column order. Random feature permutations during training and permutation ensembles encourage robustness but do not establish an exact symmetry.

At fixed depths and widths, the attention interactions in the two column stages, row refinement, row pooling and ICL scale as

$$O\Big(nmk + (2R+1)nmK + nK_p(m+K_p) + (n_c+16)^2 + n_q(n_c+16)\Big). \tag{20}$$

Retrieval adds $O(n_q n_c)$. These expressions describe attention interactions, not wall-clock time: projections and feed-forward blocks remain, and sorting context values for ECDF preprocessing costs $O(mn_c \log n_c)$, followed by $O(nm \log n_c)$ rank lookups. B4 is linear in the feature count for fixed k, R, K, K_p, but still quadratic in the context length. Section 7.7 reports the measured CPU trade-off.

3.10 Parameter budget and inference

Table 1 lists the parameter counts. B4 adds 372,032 parameters in the refinement loop and the second column stage and removes one ICL block of 546,016 parameters, for a total of 4,603,088. At inference, fitting encodes the context once and stores the column statistics, the inducing-point states of both column stages, the ICL keys and values and the decoder keys; prediction then processes query rows in independent chunks. An ensemble of E estimators averages the probabilities obtained under random permutations of feature order and class slots. By default, above 20,000 training rows, each estimator uses a stratified context subsample that retains every class. The full-context experiments in Section 9 override this limit. Section 10 describes a faster implementation of these steps that computes the same predictions up to floating-point rounding.

4 Synthetic priors

This section specifies the simulation procedure that defines $p(\mathcal{D})$ in Eq. (1). For each task we sample its dimensions and class count, generate features and labels, and split the rows into labelled context and withheld queries. Training the network of Section 3 on many such tasks teaches it to predict from a new context. The main pretraining mixture uses 90% graph v3 tasks and 10% rule tasks. The graph generator is inherited from TabICLv2 [22]; our changes concern reproducible sampling, task geometry, the v4 variant studied in ablations, and the rule-based labelling procedure. Thus the SCM approach itself is not a new contribution of this work.

4.1 Task geometry

By *geometry* we mean the task dimensions and context/query proportions. Tasks are generated in groups of eight that share their shape, which keeps padding low during training. Padding fills variable-sized tables to a common batch shape; reducing it avoids unused computation. The number of rows is log-uniform, $n \sim \mathrm{LogU}(256, 2048)$, and the context fraction is uniform in $[0.3, 0.9]$. The number of features follows the ratio m/n, drawn log-uniformly within one of three bands, $[0.01, 0.06]$ with probability 0.7, $[0.06, 0.15]$ with probability 0.2 and $[0.15, 0.5]$ with probability 0.1, then rounded and clipped to $[2, 100]$. A task is binary with probability p_{bin} and otherwise has $C \sim \mathcal{U}\{3, \ldots, 10\}$ classes. Here $\mathrm{LogU}(a, b)$ means uniform sampling of the logarithm between $\log a$ and $\log b$, $\mathcal{U}\{3, \ldots, 10\}$ is discrete uniform sampling, and p_{bin} is the probability of a binary task. The 2–10 training classes are distinct from the 16 label slots available in the architecture. Half of the tasks are resampled to an imbalanced class distribution, with a binary minority share drawn from $\mathrm{LogU}(0.01, 0.5)$ and multiclass proportions from a Dirichlet distribution. Every random draw is seeded by its coordinates (prior, shard, group, task, phase) through a hierarchical seed sequence, so shards are byte-identical for any number of worker processes.

4.2 Graph prior

The *graph prior* is a distribution over tasks whose features and target are generated from a shared random SCM, as introduced in Section 2. Our implementation uses the graph sampler, node functions and converters of TabICLv2 [22], with deterministic seeding and our own retry and shape policies. A random directed acyclic graph is sampled; each node applies a random function to its parents, chosen among MLPs, random oblivious trees, discretizations, linear and quadratic maps, Gaussian processes with product kernels and products, with sum, product, max or log-sum-exp aggregation and Kumaraswamy warping; some nodes become categorical. Features and the target are read from selected nodes. Each draft task passes an out-of-bag filter based on ExtraTrees [8], which rejects tasks whose labels cannot be predicted at all. Version 3 uses $p_{\mathrm{bin}} = 0.55$. Version 4 keeps heavy tails in most tasks, quantizes some numeric columns into categorical codes with cardinality $\mathrm{LogU}(16, 256)$ and Zipf frequencies, raises p_{bin} to 0.78, splits numeric targets at quantiles for part of the multiclass tasks and reads the target from the final node in 90% of the drafts.

4.3 Rule prior

The *rule prior* is a distribution over tasks in which an explicit sampled rule supplies the target. It draws features from graph v4 and assigns labels with a known oracle (the label-generating rule) over one to four observed driver columns. Unlike the graph prior, the target relationship is specified directly on these observed columns. Its families and probabilities are XOR (25%), parity (20%), shallow trees (20%), lookup tables (10%), products (10%), AND (7.5%) and OR (7.5%). For binary XOR and parity, the drivers are resampled independently within the partitions of their marginal distributions, with source thresholds τ_k at quantiles between 30% and 70%, and the label is

$$y = \bigoplus_{k=1}^{K} \mathbf{1}\left[x_{d_k} > \tau_k\right]. \tag{21}$$

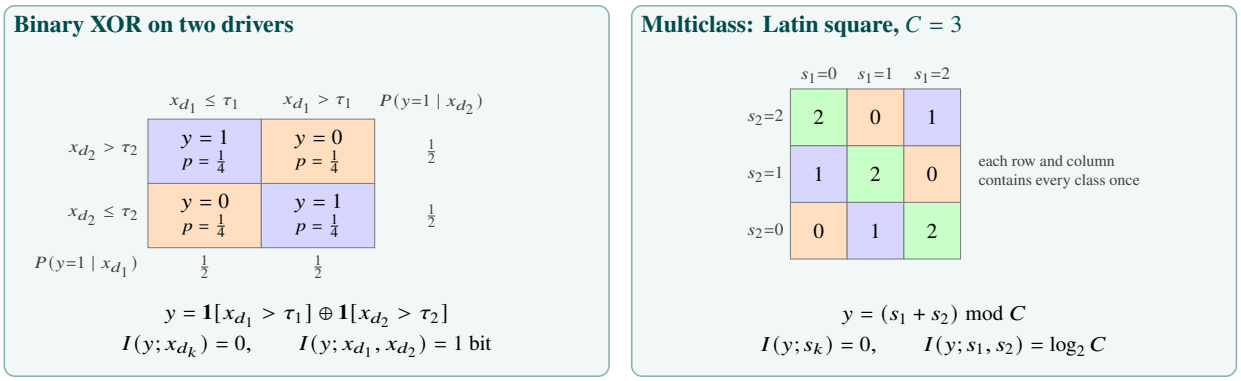


Figure 3. Interaction-only rule tasks. Left: binary XOR on two drivers whose source thresholds split balanced factorial cells, so each driver alone predicts the label at chance. Right: for C classes, symbols of two drivers are summed modulo C, a Latin square in which every row and column contains each class once.

Since the factorial cells are balanced, $P(y = 1 \mid x_{d_k}) = 1/2$ for every single driver: the signal lives entirely in the interaction. For C classes, each driver is mapped to a symbol $s_k \in \{0, \dots, C-1\}$ with uniform frequencies and $y = \left(\sum_k s_k\right) \bmod C$, a Latin-square construction with the same property (Figure 3). Tree, lookup, product, AND and OR tasks receive, in a quarter of the cases, a mixture with the logits of the underlying graph,

$$\ell = a\,\ell_{\text{rule}} + b\,\ell_{\text{graph}}, \qquad a \sim \mathcal{U}[0.75, 1.5],\ b \sim \mathcal{U}[0, 0.5], \tag{22}$$

followed by a softmax at temperature 0.125. Label noise between 0% and 15% is added, and a gate requires at least two rows per class and an agreement of at least 70% between the oracle and the noisy test labels. No fitted classifier is used to filter rule tasks.

4.4 Load-time augmentation

When a task is loaded for training, its real features are permuted, and 30% of the tasks receive missing values. The mechanism is drawn uniformly among missing completely at random (MCAR), missing at random (MAR), and missing not at random (MNAR). Respectively, the missingness probability is independent of values, depends on another feature, or depends on the value being hidden. A fraction $\mathcal{U}[0.1, 1]$ of the columns is affected, and each affected column j has a rate $\rho_j \sim \text{LogU}(0.01, 0.5)$. Under MCAR a cell is missing with probability ρ_j. Under MAR and MNAR the probability is $2\rho_j\, s_i$, where $s_i \in [0, 1]$ is the normalized mid-rank of $\pm x_{ik}$ for another column k (MAR) or for $k = j$ (MNAR), so the expected rate stays ρ_j. At least two observed context cells are kept in every column.

5 Training

5.1 Objective and optimization

Each step draws a batch $\mathcal{B}$ of tasks and minimizes the empirical version of Eq. (1), with logits of classes absent from a task masked out. In this expression t indexes tasks, and n_q^t, $\mathcal{Q}_t$ and $\mathcal{D}_{\text{ctx}}^t$ are their query counts, query index sets and contexts:

$$\hat{\mathcal{L}}(\theta) = \frac{1}{|\mathcal{B}|} \sum_{t \in \mathcal{B}} \frac{1}{n_q^t} \sum_{i \in \mathcal{Q}_t} -\log q_\theta\left(y_i \mid x_i, \mathcal{D}_{\text{ctx}}^t\right). \tag{23}$$

The two-dimensional weight matrices of all attention blocks are optimized with Muon [11], the remaining parameters with AdamW [15] ($\beta_1 = 0.9$, $\beta_2 = 0.98$, weight decay $\lambda = 0.01$). In the optimizer equations below, t indexes optimization steps. For a weight $W \in \mathbb{R}^{a \times b}$ with gradient G_t, Muon orthogonalizes the

Nesterov momentum with five Newton-Schulz iterations and rescales the update to the root-mean-square magnitude of AdamW [14]:

$$M_t = \mu M_{t-1} + G_t, \qquad N_t = G_t + \mu M_t, \qquad O_t = \mathrm{NS}_5(N_t) \approx U_t V_t^\top, \tag{24}$$

$$W_t = (1 - \eta_t \lambda)\, W_{t-1} - \eta_t \cdot 0.2\sqrt{\max(a, b)}\; O_t, \tag{25}$$

where $N_t = U_t \Sigma_t V_t^\top$ and $\mu = 0.95$. Both optimizers share the learning rate

$$\eta_s = \begin{cases} \eta_0 \dfrac{s+1}{W}, & s < W, \\ \eta_0 \Big(0.1 + 0.45\big(1 + \cos(\pi \min\{1, \frac{s-W}{S-W}\})\big)\Big), & s \geq W, \end{cases} \tag{26}$$

with $\eta_0 = 10^{-3}$, warmup W and total steps S. Gradients are clipped to norm 1, the forward pass runs in bfloat16 autocast, and an exponential moving average of the weights, $\bar{\theta} \leftarrow \beta\bar{\theta} + (1-\beta)\theta$ with $\beta = 0.999$ after warmup and 0.99 during it, provides the evaluated model.

5.2 Micro-batching

A step on one GPU consists of four groups of eight tasks. Groups are split along the task axis into micro-batches with a budget of cells (tasks × rows × features), 8×10^5 in the final run, and the gradients of the micro-batches are accumulated, each weighted by the number of tasks in the step. The long-context stage of Section 9 replaces the cell count with a measured cost that also counts the rows.

5.3 Data parallelism

Task shapes give each GPU a different number of micro-batches, so gradients are synchronized once per step. With P ranks and local task sets $\mathcal{B}_r$, the task-weighted gradient is

$$g = \frac{1}{N} \sum_{r=1}^{P} \sum_{t \in \mathcal{B}_r} \nabla_\theta\, \ell_t(\theta), \qquad N = \sum_{r=1}^{P} |\mathcal{B}_r|. \tag{27}$$

The gradients are flattened into a single buffer and summed with one all-reduce, which matches the task-weighted gradient of a single process over $\bigcup_r \mathcal{B}_r$ in exact arithmetic; the reduction order can introduce floating-point differences. Parameters unused on all ranks retain empty gradients. Two RTX 5090 GPUs process 64 tasks per step.

5.4 Chunked streaming of fresh tasks

A long run generates tasks concurrently with training rather than retaining the full stream on disk. The run is divided into K chunks of S_c steps. Chunk k holds one pool per prior p, generated with seed $\sigma_p + k$, so any chunk can be regenerated byte for byte. For a mixture weight w_p, τ tasks per step, a safety margin μ_s and Π reads of every chunk, the pool size is

$$T_p = 256 \left\lceil \frac{w_p\, S_c\, \tau\, (1 + \mu_s)}{256\, \Pi} \right\rceil. \tag{28}$$

Within each read, chunk groups are shuffled and partitioned without replacement across GPU ranks and loader workers. Repeated reads apply new feature permutations and missingness. The producer keeps at most $A = 3$ chunks ahead of the trainer and deletes a chunk after all consumers finish it (Figure 4). Checkpoints are saved every 1,000 steps; retries and chunk boundaries retain the schedule of the full $S = KS_c$-step run.

The final run uses $K = 16$, $S_c = 7{,}500$, $\tau = 64$, $\mu_s = 0.05$, $\Pi = 2$ and the mixture of graph v3 (90%) and rule v1 (10%), so each chunk holds $T_{\text{v3}} = 226{,}816$ and $T_{\text{rule}} = 25{,}344$ tasks. Over 120,000 steps the model draws 7.68 million tasks from pools containing 4.03 million distinct tasks. Generation runs on 72 CPU processes pinned to 80 physical cores and produces about 120 tasks per second, roughly twice what the trainer consumes, with 16 cores reserved for training.

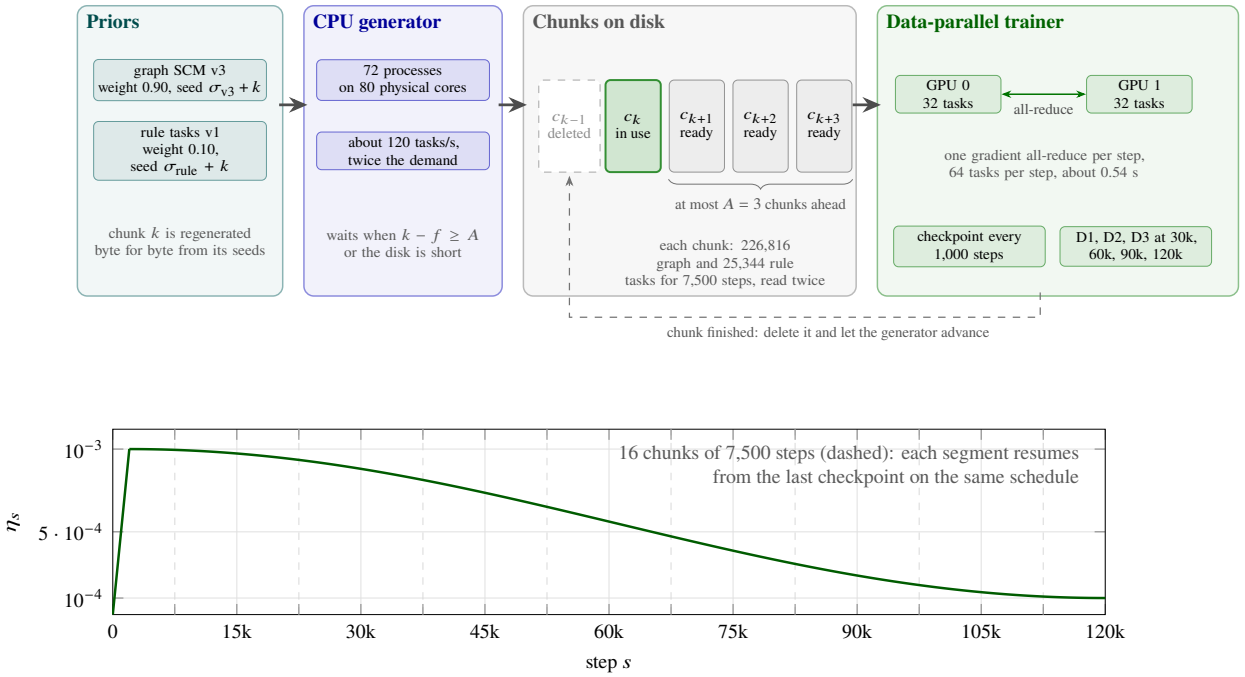


Figure 4. Training system of the final run. Top: CPU processes generate chunks of fresh tasks from the priors, at most $A = 3$ chunks ahead of the trainer; two GPUs train on the current chunk with one gradient all-reduce per step, and every finished chunk is deleted. Bottom: the learning rate over the 120,000 steps; the sixteen segments, one per chunk, resume from checkpoints and follow one warmup and cosine schedule.

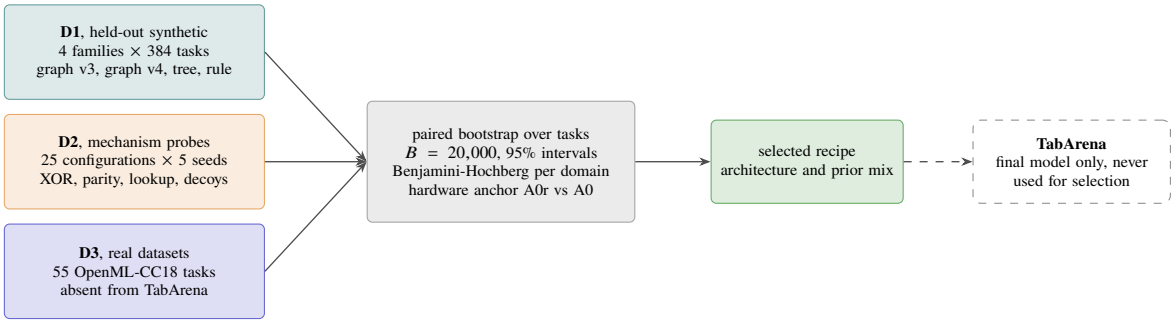


Figure 5. Selection protocol. Design decisions are taken on D1, D2 and D3 with paired statistics (the long-context stage also on learning curves of the largest D3 datasets, Section 9); TabArena is reserved for the final models.

6 Evaluation protocol

Variants are compared on three sets that are disjoint from TabArena (Figure 5).

D1, held-out synthetic tasks. Four families of 384 tasks each, graph v3, graph v4, tree and rule, generated with a dedicated seed that differs from every training seed. Since the rule family weighs 25% of the D1 mean and only 10% of the final training mixture, we read the families separately and use the graph and tree families as guards.

D2, mechanism probes. Twenty-five probe configurations with five seeds each, 800 context rows and 1,000 test rows, where the informative columns are placed among Gaussian distractors to reach d features. Probes include XOR of two features, parity of three, a depth-two tree rule, a sparse linear control, lookup tables with 16 and 64 categories, heavy-tailed thresholds, ratios, and XOR with near-duplicate

decoy columns.

D3, real datasets. The 55 classification tasks of OpenML-CC18 [2] with at most ten classes that are absent from TabArena, by name or by matching row and class counts, excluding image datasets. Each task is reduced to at most 1,000 rows and 100 random features and evaluated by stratified five-fold cross-validation; AUC is averaged over folds, using one-vs-rest macro AUC for multiclass tasks.

Paired statistics. For two models A and B evaluated on the same n tasks with scores a_t and b_t, we report $\Delta = \frac{1}{n}\sum_t (a_t - b_t)$ with a percentile interval from $B = 20{,}000$ bootstrap resamples of the tasks. Each resample draws n task indices with replacement and uses the same indices for both models; Δ_b^* is the mean difference in resample b. We report the two-sided p-value

$$p = \frac{1 + \#\{b : \ |\Delta_b^* - \Delta| \geq |\Delta|\}}{B + 1}. \tag{29}$$

For D2, with five seeds per probe, the bootstrap enumerates all resamples and a sign-flip test gives the p-value. Within every domain and metric, p-values are adjusted with the Benjamini-Hochberg procedure [1], a multiple-testing adjustment targeting the expected proportion of false discoveries among rejected hypotheses, rather than the error rate of each test separately. All ablations use a single training seed, so the intervals capture task sampling and leave out the variance across training runs; Section 7.2 estimates the latter through one hardware anchor rather than a repeated-seed variance estimate. An interval that includes zero does not establish equivalence. Unless otherwise stated, an AUC difference in points means 100 times the difference on the $[0, 1]$ AUC scale. D1–D3 are selection sets, so their results are not an independent final test of choices made with those sets.

7 Experiments

7.1 Setup

Ablations train for 8,000 steps of 32 tasks with warmup 800 and seed 0, which amounts to 256,000 task draws. Series A and B ran on an AMD RX 7900 XT, series C on an RTX 5090. Unless stated otherwise, the models are evaluated with one estimator. The runs are: A0 (baseline architecture, graph v3 only); B1 (radial basis cell embedding); B2 (auxiliary masked-cell reconstruction loss, weight 0.1); B3 (row summary mode); B4 (B3 with row refinement and one ICL block less); A2 (A0 trained on graph v4 85% and rule 15%); A3 (A2 with a stronger load-time augmentation); A0r (A0 repeated on the RTX 5090); C1 and C2 (B3 and B4 trained on graph v3 90% and rule 10%); C3a and C3b (B4 and B3 trained on graph v3 45%, graph v4 45% and rule 10%).

7.2 How large is a single-seed difference?

A0r repeats the recipe of A0 on a different GPU and software stack, which changes the floating-point order of operations and therefore the training trajectory. On D1 the two runs agree within 0.06 points. On D3, A0r scores 0.31 points higher with an interval that excludes zero (Table 3). The external suite has only 55 tasks, and a change of trajectory alone moves its mean by about 0.3 points in this comparison. We therefore interpret single-seed D3 differences of that size cautiously and give more weight to D1, whose 1,536 tasks yield narrower task-bootstrap intervals.

7.3 Architecture

Among the architecture variants, only B4 produces a clear gain (Table 3). It improves the D1 mean by 0.56 points over A0, with the largest effects on graph v4 (+0.95) and tree tasks (+1.10), and improves D3 by 0.51 points, mostly on multiclass tasks (+0.71). The gain over B3 on D1 (+0.44) concentrates on the same two families, while the D3 gain over B3 stays within the noise level established above. The radial basis embedding

		D1 (held-out synthetic)					D3
Run	Change	graph v3	graph v4	tree	rule	mean	real
A0	baseline	.8755	.8256	.8308	.7322	.8160	.8869
A0r	A0 on another GPU	.8756	.8250	.8331	.7281	.8155	.8900
B1	RBF cell embedding	.8751	.8262	.8366	.7337	.8179	.8897
B2	masked-cell auxiliary loss	.8708	.8242	.8323	.7220	.8123	.8870
B3	row summary	.8779	.8269	.8325	.7317	.8173	.8897
B4	B3 + row refinement	.8794	.8351	.8418	.7302	.8216	.8921
A2	graph v4 85% + rule 15%	.8643	.8098	.8180	.7993	.8228	.8838
A3	A2 + augmentation	.8627	.8177	.8132	.7691	.8157	.8819
C1	B3, v3 90% + rule 10%	.8755	.8251	.8317	.7751	.8269	.8859
C2	B4, v3 90% + rule 10%	.8791	.8340	.8408	.8236	.8444	.8937
C3a	B4, v3/v4 45% + rule 10%	.8784	.8301	.8402	.7721	.8302	.8901
C3b	B3, v3/v4 45% + rule 10%	.8731	.8237	.8296	.7692	.8239	.8864

Table 2. Mean AUC of all ablations after 8,000 steps, one estimator. C2 is the recipe of the final run.

Comparison	D1 mean	D1 rule	D3
A0r − A0	−0.06 [−0.22, 0.11]	−0.41 [−0.85, 0.01]	+0.31 [0.06, 0.54]
B4 − A0	+0.56 [0.34, 0.78]	−0.20 [−0.86, 0.48]	+0.51 [0.18, 0.84]
B4 − B3	+0.44 [0.23, 0.64]	−0.15 [−0.74, 0.47]	+0.24 [−0.18, 0.62]
A2 − A0	+0.68 [0.25, 1.13]	+6.71 [5.43, 8.03]	−0.32 [−0.82, 0.18]
C1 − B3	+0.96 [0.66, 1.27]	+4.35 [3.35, 5.38]	−0.38 [−0.78, −0.05]
C2 − B4	+2.27 [1.82, 2.74]	+9.34 [7.78, 10.94]	+0.17 [−0.15, 0.44]
C2 − C1	+1.75 [1.42, 2.10]	+4.84 [3.73, 6.02]	+0.78 [0.39, 1.18]
C3a − C2	−1.41 [−1.78, −1.08]	−5.14 [−6.40, −3.95]	−0.37 [−0.61, −0.15]
C3b − C1	−0.30 [−0.50, −0.11]	−0.60 [−1.18, −0.05]	+0.05 [−0.30, 0.42]

Table 3. Paired differences in AUC points with 95% bootstrap intervals over tasks (1,536 D1 tasks, 384 rule tasks, 55 D3 tasks).

of B1 helps only the tree family. The auxiliary masked-cell loss of B2 gives no detectable D3 gain (0.8870 against 0.8869) and lowers the D1 mean; at this training budget, we find no benefit for classification. The summary row mode of B3 gives no clear accuracy difference from the baseline while cutting the training time per step by 30%. The B4 comparison changes three components together: row refinement, a second column stage and ICL depth (eight to seven). The experiments identify the effect of this configuration, rather than the isolated contribution of each added module.

7.4 Priors and the interaction between prior and architecture

Adding rule tasks teaches interactions, and how much the model learns depends on the row stage. With graph v4 and 15% rule tasks, A2 gains 6.7 points on the rule family and loses between 1.1 and 1.6 points on every other D1 family. Restricting the change to 10% rule tasks on graph v3 removes most of that cost: C2 improves on B4 by 9.3 points on rule tasks while the graph and tree families move by at most 0.11 points in either direction, all within their intervals. The same mixture on the summary architecture (C1) gains 4.4 points on rule tasks over B3, half of what B4 gains, and C2 beats C1 by 1.75 points on D1 and by 0.78 points on D3.

Figure 6 shows the same effect on the probes. With rule tasks, C2 solves XOR among 10 and 30 features, three-way parity among 10 features and XOR with near-duplicate decoys among 30 features (0.998, against 0.559 for A0), all with AUC of 0.97 or more. C1 and A2, which see rule tasks with different architectures or mixtures, stay far below. The controlled C2–C1 comparison supports an interaction between rule data and

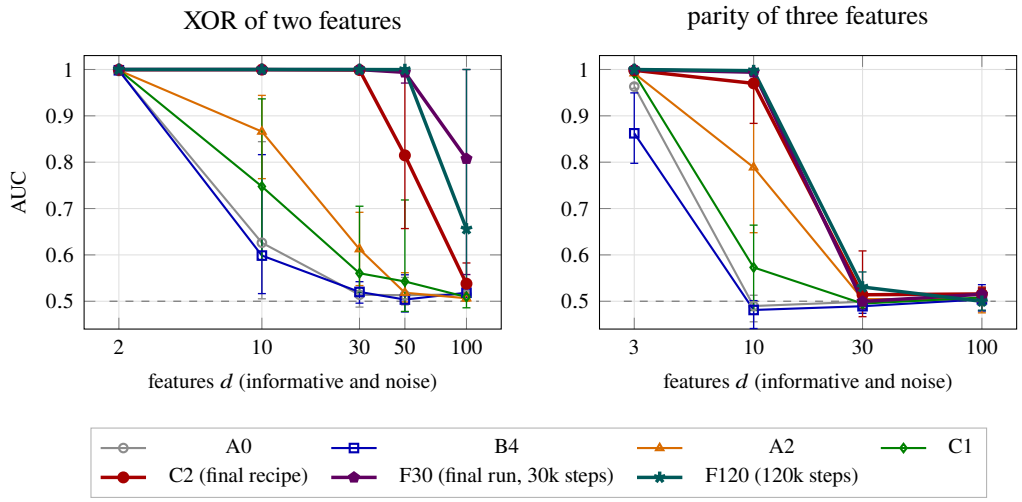


Figure 6. D2 probes: AUC as the number of features d grows, with two (XOR) or three (parity) informative features and Gaussian noise in the others. Points are means over five seeds, bars span the lowest and highest seed, the dashed line is chance. Among the ablations, only C2, which combines the row refinement with rule tasks, solves XOR up to 30 features and parity up to 10. F30 and F120, the final run after 30,000 and 120,000 steps (Section 8), extend XOR to 50 features; XOR among 100 features varies between checkpoints (0.81 for F30, 0.66 for F120) and parity among 30 features stays close to chance.

the B4 configuration: richer cell processing before compression is a plausible explanation, but the row and column refinement contributions are not isolated. The probes also mark the limit of this skill: C2 still reaches 0.82 on XOR among 50 features, while parity among 30 features and XOR among 100 stay at chance for every model (0.51 and 0.54 for C2).

Replacing half of the graph v3 tasks with graph v4 hurts (C3a against C2): the D1 mean drops by 1.41 points, mostly on rule tasks, D3 drops by 0.37 points with an interval below zero, and the probe gains disappear. Graph v4 tasks are harder and more balanced, and the checkpoint trend of A2 suggests that they are learned more slowly; at our budget they compete with skills that the other components already provide. The final mixture therefore contains no graph v4 tasks.

7.5 Training dynamics

Figure 7 follows three runs across checkpoints. The lead of B4 over A0 is large early in training, 8.3 points of D1 at 2,000 steps, and settles at about half a point by 8,000 steps. A2 trails A0 at 4,000 steps (0.7731 against 0.8028) and then improves faster, with its rule AUC still rising by 2.6 points between 6,000 and 8,000 steps. Both observations indicate that 8,000 steps rank only the clear effects and that some components may need much longer training to show their value, which motivates the long final run.

7.6 Comparison with gradient-boosted trees

Table 4 places LightPFN among default tree ensembles on D3, with the same paired bootstrap as the ablations. After 256,000 synthetic tasks and with a single estimator, C2 has a higher mean AUC than default random forest [3], LightGBM [12] and XGBoost [4], but none of the corresponding bootstrap intervals excludes zero (+0.04, +0.30 and +1.19 points), and it is 0.86 points behind default CatBoost [19] ($[-1.70,\ -0.19]$), with the higher AUC on 27% of the tasks. The final run closes and then reverses that gap. Its checkpoint after 30,000 steps, F30 (Section 8), has no clear mean AUC advantage over CatBoost: 0.31 points higher, with an interval of $[-0.13,\ 0.73]$ that includes zero. The final checkpoint after 120,000 steps, F120, is 0.81 points above CatBoost ($[0.39,\ 1.29]$) with the higher AUC on 76% of the tasks, on binary (+0.72) as well as

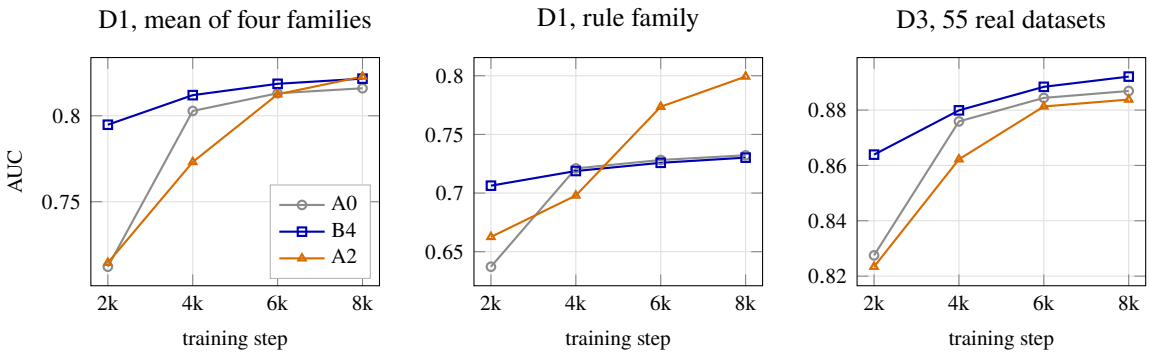


Figure 7. AUC of the EMA checkpoints after 2,000 to 8,000 steps of the ablations. B4 leads early and keeps about half a point at the end; A2 (graph v4 and rule tasks) starts slower and is still rising on the rule family when the ablation stops.

Model (D3, 55 tasks)	mean AUC	F120 − model	F120 higher
LightPFN F120, final run at 120,000 steps	.9104		
LightPFN F30, final run at 30,000 steps	.9054	+0.49 [0.17, 0.89]	84%
CatBoost, default	.9023	+0.81 [0.39, 1.29]	76%
LightPFN C2, 8,000 steps	.8937	+1.66 [1.03, 2.46]	95%
Random forest, default	.8933	+1.70 [1.02, 2.51]	85%
LightGBM, default	.8907	+1.96 [1.11, 3.07]	91%
XGBoost, default (54 successful tasks)	.8889	+1.99 [1.28, 2.84]	93%
LightPFN A0, 8,000 steps	.8869	+2.34 [1.72, 3.05]	98%
XGBoost, including failure penalty	.8819	+2.85 [1.44, 5.01]	93%

Table 4. External suite D3 against default tree ensembles: one estimator for LightPFN, same splits for every model. Differences in AUC points with 95% bootstrap intervals; the last column is the share of tasks on which F120 has the higher AUC. All rows use 55 tasks except the successful-task XGBoost comparison, which excludes *sick* from both models and has F120 mean AUC 0.9088. The last row shows the 55-task result when the wrapper failure receives uniform predictions.

multiclass tasks (+0.89), both with intervals above zero. F120 leads default random forest, LightGBM and XGBoost by 1.70, 1.96 and 2.85 points, with intervals above zero and the higher AUC on 85 to 93% of the tasks. The 2.85-point XGBoost difference includes a failed run: on all five folds of the OpenML task *sick* (task 3021), the categorical-input wrapper creates a categorical column with no observed training categories, which XGBoost rejects. The harness assigns uniform predictions to the failed folds; their AUC of 0.5 is a failure penalty, not a fitted model's predictive performance. On the 54 successful tasks, the paired lead of F120 is 1.99 points [1.28, 2.84], with means 0.9088 for F120 and 0.8889 for XGBoost. We use this successful-task comparison for the predictive claim and retain the penalized result for transparency in Table 4. All D3 tasks have at most 1,000 training rows; Section 9 turns to larger tables. An earlier model of the series, r2 (baseline architecture, 15,000 steps on graph v3), had been evaluated on the TabArena lite protocol before the selection protocol was introduced: with four estimators it reached a mean AUC of 0.8290 against 0.8294 for default CatBoost, ahead of default LightGBM (0.8104) and XGBoost (0.7966). We report it as a reference point and have not used TabArena for any decision since; Section 11 evaluates the final models.

7.7 Cost

On an RTX 5090, a training step of 32 tasks takes a median of 0.35 s for A0, 0.24 s for B3 trained as C1 and 0.52 s for B4 trained as C2. On the CPU (AMD EPYC 9654, 16 threads, one estimator, shapes from 800 context rows and 22 features up to 10,000 rows and 30 features), the fit and predict time of B3 is 0.60 to 0.76

times that of A0 and the time of B4 is 1.24 to 1.38 times. We chose B4 because its accuracy gain over B3 (0.8 points of D3 and 1.75 points of D1 with the final mixture) outweighs a CPU cost about twice as high. Section 9 reports timings for large tables on a shared server, Section 10 a faster implementation of inference with the same predictions up to floating-point rounding, and Section 11 the cost on TabArena against the tree ensembles.

8 Final run

The final model uses the C2 recipe: B4, graph v3 at 90% and rule v1 at 10%, no augmentation beyond feature permutation and missingness. It trains for 120,000 steps of 64 tasks on two RTX 5090 GPUs with the data-parallel scheme and the chunked stream of Section 5, warmup 2,000 steps and a cosine schedule over the full run. A 300-step, three-chunk pilot verified recovery after an interrupted training process, chunk deletion and schedule continuity. The full run took about 18 hours at 0.52–0.56 s per step. Checkpoints at 30,000, 60,000, 90,000 and 120,000 steps (F30, F60, F90 and F120) were evaluated on D1, D2 and D3. TabArena was used only after the long-context stage had been selected (Section 11).

8.1 Checkpoints from 30,000 to 120,000 steps

After 30,000 steps, 1.92 million task draws and a quarter of the run, with the learning rate still at 88% of its peak, F30 improves on C2 in every family of D1 and on D3 (Table 5). This comparison measures the progress of the run rather than a design choice: F30 has seen 7.5 times as many tasks as C2, with a batch twice as large and fresh data. The largest gain is on rule tasks, 6.65 points, followed by graph v3 (1.40), graph v4 (1.24) and tree tasks (1.10); on D3 the gain is 1.17 points.

The later checkpoints keep improving, with shrinking returns. The D1 mean rises from 0.8703 to 0.8746, 0.8773 and 0.8781, and D3 from 0.9054 to 0.9086, 0.9101 and 0.9104: on D3 each further quarter of the run adds 0.32, 0.15 and 0.03 points. Between F60 and F120 every D1 family still gains, by 0.26 to 0.56 points with intervals above zero, and D3 gains 0.18 points $[0.05,\ 0.32]$. Over the whole run F120 improves on C2 by 3.38 points on D1 and 1.66 points on D3, with the higher AUC on 95% of the external tasks. The training loss, averaged over the last 500 steps before each checkpoint, goes from 0.398 to 0.386, 0.372 and 0.367. These diminishing gains do not establish convergence beyond this training budget; the training tables have at most 2,048 rows.

The probes move the limit of the interaction skill outward (Figure 6). XOR among 50 features rises from 0.815 for C2 to 0.994 at F30 and 1.000 at F120, three-way parity among 10 features from 0.970 to 0.997, and the lookup table with 64 categories among 100 features from 0.858 to 0.896. Parity among 30 features stays close to chance (0.50 to 0.54 across the checkpoints) and among 100 at chance. XOR among 100 features is unstable: 0.807, 0.914, 0.825 and 0.656 at the four checkpoints, with a wide spread over the five seeds, at the edge of what the model has learned.

9 Large tables

9.1 Learning curves on real datasets

D3 reduces each dataset to at most 1,000 total rows before cross-validation, and pretraining tables have at most 2,048 rows, so neither shows how the model uses a long context. The scale test takes the 15 D3 datasets with at least 5,000 rows. For each dataset and each of two repeats it draws a stratified test set of $\min(2000, n/5)$ rows and, from the remaining rows, stratified training sets of 1,000, 2,000, 4,000, 8,000, 16,000 and 32,000 rows where the dataset is large enough (15, 15, 15, 9, 6 and 6 datasets), plus one with all remaining rows (4,324 to 94,320, median 8,992); at most 100 random features are kept, as in D3. LightPFN reads the whole training set as context (full context) or, as in pretraining, stratified subsamples of 2,048 rows, averaged over eight estimators. The tree ensembles use default settings. Differences are averaged over the repeats of each dataset and bootstrapped over datasets.

	D1 (held-out synthetic)					D3
	graph v3	graph v4	tree	rule	mean	real
C2, 8,000 steps × 32 tasks	.8791	.8340	.8408	.8236	.8444	.8937
F30, 30,000 steps × 64 tasks	.8930	.8464	.8518	.8900	.8703	.9054
F60, 60,000 steps	.8968	.8526	.8522	.8969	.8746	.9086
F90, 90,000 steps	.8991	.8545	.8540	.9014	.8773	.9101
F120, 120,000 steps	.8994	.8558	.8549	.9025	.8781	.9104
F120 − C2	+2.03	+2.18	+1.41	+7.89	+3.38	+1.66
95% interval	[1.59, 2.50]	[1.75, 2.65]	[1.09, 1.74]	[6.41, 9.40]	[2.95, 3.82]	[1.03, 2.46]
F120 − F60	+0.26	+0.32	+0.27	+0.56	+0.35	+0.18
95% interval	[0.09, 0.45]	[0.13, 0.52]	[0.05, 0.52]	[0.24, 0.95]	[0.23, 0.49]	[0.05, 0.32]

Table 5. Checkpoints of the final run (EMA weights, one estimator) against C2, the 8,000-step ablation with the same recipe. Mean AUC and paired differences in AUC points with 95% bootstrap intervals over tasks.

		CatBoost	AUC − default CatBoost (points)							
rows	sets	AUC	F30	F120	95% interval	F120×8	LC11k	LC39k	95% interval	LGBM / XGB
1,000	15	.8959	−0.38	+0.26	[−0.13, 0.72]	+0.42	+0.42	+0.46	[0.13, 0.95]	−0.01 / −0.20
2,000	15	.9071	−0.71	+0.10	[−0.27, 0.46]	+0.32	+0.37	+0.51	[0.22, 0.84]	+0.05 / −0.14
4,000	15	.9187	−1.14	−0.36	[−0.78, 0.03]	−0.40	+0.20	+0.33	[−0.08, 0.74]	+0.20 / +0.04
8,000	9	.9085	−1.62	−0.64	[−1.33, −0.14]	−0.95	+0.05	+0.15	[−0.42, 0.74]	+0.25 / +0.19
16,000	6	.8723	−3.11	−1.48	[−2.82, −0.58]	−2.14	−0.38	−0.38	[−1.70, 0.92]	+0.10 / −0.12
32,000	6	.8784	−3.52	−1.92	[−3.41, −0.86]	−2.72	−0.43	−0.42	[−1.41, 0.70]	+0.18 / +0.36
all	15	.9313	−1.81	−0.77	[−1.63, −0.09]	−1.34	+0.23	+0.31	[−0.23, 0.86]	−0.01 / +0.10

Table 6. Scale test against default CatBoost. F30 and F120: checkpoints of the final run with full context, and F120×8 averaged over eight subsamples of 2,048 rows; LC11k and LC39k: the long-context stage after 11,000 steps and at its end (39,250 steps), full context; LGBM and XGB: default LightGBM and XGBoost. Each dataset is the mean of two repeats; intervals bootstrap the datasets.

With full context, the mean AUC difference between F120 and default CatBoost is inconclusive at 1,000 and 2,000 training rows and F120 falls behind as the training set grows (Figure 8, Table 6): by 0.36 points at 4,000 rows, 0.64 at 8,000, 1.48 at 16,000 and 1.92 at 32,000, with intervals below zero from 8,000 rows, and by 0.77 points with all rows. The deficit has halved since F30 (−3.52 points at 32,000 rows), so longer pretraining on short tables also improves the use of long contexts, but it does not remove the gap. The mean differences of LightGBM and XGBoost from CatBoost have intervals that include zero at every size. From 4,000 rows up, the full context beats eight subsamples of 2,048 rows, so the model extracts information from rows beyond its pretraining length, but it gains less from them than the trees do. Where the training set fits in 2,048 rows, the eight estimators differ only in their feature orders and label slots and add 0.16 to 0.22 points. On 24 threads of a shared AMD EPYC server, fit and predict with full context take a median of 3.8 s at 16,000 rows and 11.6 s at 32,000, against 7.7 and 14.0 s for CatBoost; the in-context attention grows quadratically with the context.

9.2 Long-context stage

As in the second stage of TabICLv2 [22], training continues on longer tables from the final checkpoint, with its weights, moving average and optimizer state, a new warmup of 200 steps and a learning rate of 10^{-4} decaying with a cosine to 10^{-5}. The mixture holds about a third of the tasks from each of three pool families, all with graph v3 at 90% and rule v1 at 10%: the pretraining pools (256 to 2,048 rows), kept so that the stage does not forget the regime in which F120 beats CatBoost; medium tables of 1,024 to 10,240 rows in groups of

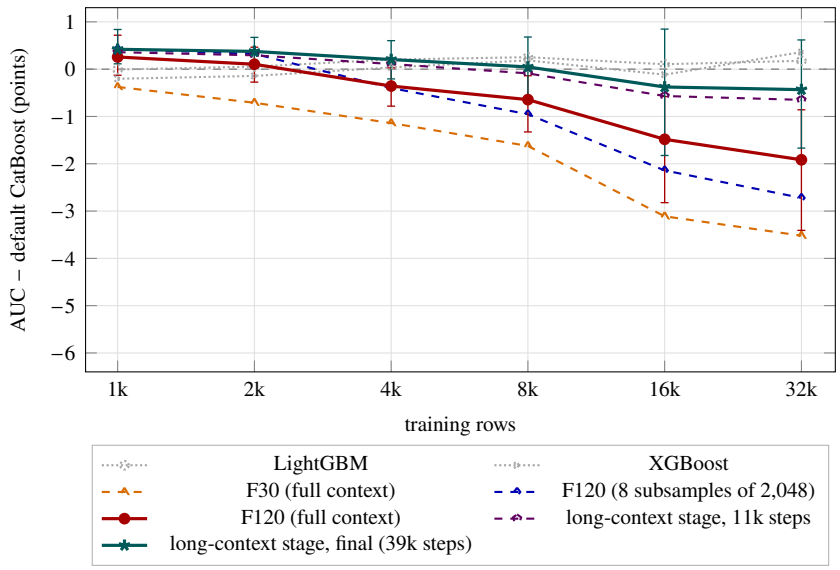


Figure 8. Scale test on the 15 D3 datasets with at least 5,000 rows: AUC difference to default CatBoost against the number of training rows (15 datasets up to 4,000 rows, 9 at 8,000, 6 at 16,000 and 32,000). Bars: 95% bootstrap intervals over datasets. The long-context stage removes most of the gap of the pretrained model F120: after 11,000 steps (tables of up to 10,240 rows, then up to 60,000) and at its end after 39,250 steps. The curve of the final checkpoint is close to the one at 11,000 steps.

four; and long tables of 4,096 to 60,000 rows in groups of two. For the medium and long tables the number of features is drawn log-uniformly between 2 and 100, independently of the rows, since the pretraining geometry would give almost every long table 100 features; the long tables also cap rows times features at 800,000, so they are narrow, as long real tables often are. By groups, the mixture weights are 15%, 35% and 50%. The long tables are generated while the stage trains, and training restarts every 2,500 steps to read the new ones, so it sees mostly fresh tables.

Long tables require row attention to be split into chunks of at most 32,768 rows to avoid the CUDA kernel's 65,535-block grid limit. They also require a micro-batch budget that includes row overhead. A forward-and-backward memory probe gives

$$M(n, m) \approx 0.09 + 1.77 \times 10^{-5}\, n\, (m + 7.65) \text{ GB} \tag{30}$$

for n rows and m features. Groups are therefore split by $n(m + 8)$ with a budget of 1.1×10^6 (about 20 GB), counting real rather than padded features. Tasks above the budget are subsampled, preserving the context fraction and stratifying context labels.

A kernel failure triggered a fallback to pretraining and medium pools for the first 7,000 steps; only the remaining steps use all three families. A loader audit over 800 groups recovered task shares of 32, 40 and 28%, with long tables reaching 59,762 rows (median 16,307); the row cap removed no rows in that audit. The stage was extended twice to 39,250 steps. Extensions stretched the remaining cosine schedule and raised the restart learning rate by up to 3.4×10^{-5}; medium tables were generated further to keep reuse near two reads. These changes are part of the training recipe evaluated here.

Results. After its first 7,000 steps, on tables of at most 10,240 rows, the checkpoint LC7k improves on F120 at every size of the scale test: by 0.47 points at 4,000 rows [0.17, 0.82], 0.55 at 8,000, 0.91 at 16,000 and 1.27 at 32,000 [0.55, 1.96], and by 0.78 points with all rows (15 of 15 datasets). The largest gain is at 32,000 rows, three times the longest table seen at that point. After 11,000 steps, the last 3,750 of them with long tables, LC11k gains more, and the final checkpoint (LC39k, 39,250 steps, the last of the run, not

rows	sets	CatBoost AUC	LC39k AUC	F120 − CatBoost	LC39k − CatBoost	95% interval	LGBM / XGB − CatBoost
10,000	13	.8141	.8168	−0.65	+0.28	[−0.53, 1.20]	−4.33 / −3.30
25,000	13	.8225	.8234	−1.03	+0.09	[−0.81, 1.08]	−3.52 / −2.72
50,000	12	.8311	.8291	−1.53	−0.20	[−1.17, 0.76]	−4.01 / −2.78
100,000	8	.8027	.7993	−2.20	−0.34	[−1.95, 1.29]	−0.83 / −2.51

Table 7. D4, large datasets outside TabArena and D3: AUC difference to default CatBoost in points, mean over datasets with a 95% bootstrap interval; the AUC columns are means over datasets. F120: the pretrained model before the long-context stage; LC39k: its final checkpoint; LGBM and XGB: default LightGBM and XGBoost.

chosen among checkpoints) has about the same curve: against F120 it is better by 0.20 points at 1,000 rows, 0.41 at 2,000, 0.69 at 4,000, 0.79 at 8,000, 1.10 at 16,000, 1.50 at 32,000 [0.47, 2.59] and 1.08 with all rows [0.43, 1.79]. Against CatBoost it is above at 1,000 and 2,000 rows (+0.46 and +0.51, intervals above zero), with intervals that include zero from 4,000 to 32,000 rows (−0.42 at 32,000) and 0.31 points above it with all rows (Table 6). The checkpoints from 11,000 to 39,250 steps differ by at most 0.4 points at 16,000 and 32,000 rows, with six datasets; most of the gain came in the first 11,000 steps and the remaining 28,000 steps, with the decay of the learning rate to 10^{-5}, add little. On the short tables no clear change is detected: LC39k differs from F120 on D1 (0.8783 against 0.8781, +0.01 points, [−0.07, 0.10]) and on D3 (0.9109 against 0.9104, +0.06 points, [−0.08, 0.21]), where it is 0.86 points above default CatBoost [0.42, 1.40], 2.02 above LightGBM and 1.76 above random forest, all with intervals above zero. Against XGBoost on the 54 successful tasks, it leads by 2.06 points [1.32, 2.94]; the 55-task comparison including the wrapper failure penalty gives 2.91 points and is not used as the predictive claim. XOR among 100 features rises from 0.66 to 0.81 (0.88 at 11,000 steps; five seeds, unstable at the edge of the trained width), XOR among up to 50 features and parity among 10 stay solved, and parity among 30 stays at chance.

The all-rows condition includes six datasets with 32,000 to 94,000 training rows. LC39k beats the three tree ensembles on *electricity* (0.979 against 0.959 for CatBoost, 0.953 for LightGBM and 0.974 for XGBoost), matches them on *nomao*, trails CatBoost by 0.7 and 0.4 points on *adult* and *jungle-chess*, and by 2.1 points on *connect-4* with 66,000 rows, where it gained 3.5 points over F120; *numerai28.6* is close to chance for every model. Fit and predict take 75 s at 94,000 rows against 10 s for CatBoost, and 57 s against 365 s on the three-class *connect-4*.

Large datasets outside the selection sets. D4 (`lightpfn.eval.large`) takes 13 OpenML datasets of 50,000 to 2.2 million rows from the AMLB and Grinsztajn suites that are neither in TabArena nor in D3 (covertype, airlines, albert, jannis, volkert, shuttle, MiniBooNE, kick, okcupid-stem, sf-police-incidents, porto-seguro, Higgs, road-safety). Each model is trained on 10,000, 25,000, 50,000 and 100,000 rows where the dataset is large enough (13, 13, 12 and 8 datasets) and tested on 5,000 rows, one repeat, with the tree ensembles at default settings (Table 7). F120 is below CatBoost at every size and significantly so from 50,000 rows (−1.53 points, −2.20 at 100,000). The final long-context checkpoint is better than F120 on every dataset at every size: by 0.93 points at 10,000 rows, 1.13 at 25,000, 1.33 at 50,000 and 1.86 at 100,000 rows [1.02, 2.90], so the gain grows with the training set, up to a size beyond the longest table seen in training (60,000 rows). Against CatBoost the interval of the mean difference includes zero at every size, with higher AUC on 7 of 13, 5 of 13, 6 of 12 and 3 of 8 datasets. It leads on *porto-seguro* (+3.0 to +4.2 points), *jannis*, *covertype* and *Higgs* and is level on *MiniBooNE* and *shuttle*; it trails on *albert* (−1.6 to −3.7), *road-safety* (−1.9 to −3.6), *kick* (−1.0 to −2.0) and, at 50,000 rows and more, *sf-police-incidents* and *airlines*. Default LightGBM and XGBoost have a mean AUC 2.6 to 4.6 points below it up to 50,000 rows; at 100,000 rows XGBoost is 2.2 points below it and LightGBM, 0.5 points below, is closer. Fit and predict take a median of 4.4, 14, 40 and 106 s at 10,000 to 100,000 rows on 24 threads of the shared server, against 21, 31, 40 and 74 s for CatBoost at the same settings.

The 15 datasets of the scale test also motivated the stage, but D4 did not enter its design, and the final checkpoint is the last one of the run; TabArena remains the independent test.

10 Inference

Fitting encodes the context of each estimator once; prediction passes the query rows through the cell stages and attends to the cached context (Section 3). Every result up to Section 9, and the initial TabArena evaluations of Section 11, use an implementation that runs each stage on the whole table at once. Three changes lower the cost without changing the function that is computed. They are the defaults of the classifier, and `chunk_cells=None, batch_cells=0, fold=False` restores the earlier path.

Cache-blocked cell stages. The column stages treat every column independently, and row refinement and row compression treat every row independently. A table of $n \times m$ cells can therefore be processed in pieces: groups of whole columns for the column stages and groups of whole rows for the row stages, written into one preallocated buffer, with the inducing-point caches of the column stages reassembled in their original order. With pieces of about 1,024 cells per CPU thread, the activations of a piece stay in the processor caches instead of streaming the whole table through memory at every operation. Each column and each row receives the same inputs as before; only the blocking of the matrix products changes. The in-context stage, which needs all context rows together, is unchanged. On the GPU the pieces hold about one million cells and only cap the memory.

Folded copy. The classifier keeps a copy of the network in which the weight of every RMSNorm is multiplied into the linear map that reads its output: the query, key-value and first feed-forward projections of every block, and the query and key projections of the decoder after the last in-context norm. The normalization left behind has no parameters. In the blocks with rotary embeddings, the query and key dimensions of every head are permuted so that each rotation acts on adjacent pairs and becomes one complex multiplication. The copy computes the same function with a different order of floating-point operations.

Estimators batched on the GPU. On the GPU, the E estimators of a fit are encoded and queried as one batch of up to about four million cells. On the CPU they stay sequential: once the pieces fit in cache, batching gave no further gain. The random draws of each estimator (context subsample, feature order, label slots) follow the same sequence as before, which a test checks, and the probabilities are still summed estimator by estimator in double precision.

Equivalence. Tests compare each path with the plain one on randomly initialized networks, with missing values and constant columns: cache blocking and the folded copy in five configurations (logits within $2 \cdot 10^{-5}$), cache blocking with zero-padded features and with double-precision inputs, and the classifier with seven combinations of the three changes in two configurations (probabilities within 10^{-6}, same predicted classes). On the first fold of the 38 TabArena classification tasks at full size, cache blocking alone returns bitwise identical probabilities on 36 tasks and differences below $6 \cdot 10^{-8}$ on the other two. On the GPU, batching and folding together change the probabilities by at most $6.8 \cdot 10^{-6}$, with the same predicted class in every row. Both TabArena protocols were rerun with the new defaults, on the CPU and on the GPU (Section 11): every rank is the same to the fourth decimal.

Speed and memory. Table 8 times binary synthetic tables with four estimators on eight threads pinned to eight cores of the EPYC 9654 server, one job at a time, best of two repeats. Fit plus predict takes 1.1 to 4.1 times less time and peak memory falls by up to 2.3 times. Cache blocking gives most of the gain and the folded copy adds 2 to 14%. The gain grows with the number of cells per row and is largest with 200 features, where the plain cell stages are limited by memory traffic. With AVX2 kernels, the instruction set of many laptop CPUs, the speedups are 1.07 to 4.0. On long tables with few features most of the prediction time is the attention of the query rows over the cached context, which the changes leave as it is: with 20,000 context rows,

	CPU, 8 threads: fit + predict (s)				peak memory (GB)		GPU (ms)
train × features × test	plain	fast	speedup	AVX2	plain	fast	plain / fast
500 × 10 × 250	0.34	0.31	1.10	1.07	0.75	0.76	64 / 20
1,000 × 50 × 500	2.14	1.25	1.71	1.53	0.95	0.82	63 / 34
3,000 × 30 × 1,500	5.12	2.92	1.75	1.74	1.10	0.85	105 / 83
2,500 × 200 × 1,250	41.7	10.2	4.08	4.00	2.07	1.17	334 / 293
10,000 × 20 × 5,000	24.1	13.9	1.73	1.65	1.40	1.01	446 / 372
20,000 × 100 × 2,000	151.7	53.8	2.82	2.75	5.94	2.53	1,589 / 1,480

Table 8. Inference cost on binary synthetic tables with four estimators. CPU: AMD EPYC 9654, eight threads on eight cores, one job at a time, best of two repeats, AVX-512 kernels; plain is the implementation used for every earlier result, fast the new defaults (cache blocking and folded copy); AVX2 is the same speedup with AVX2 kernels on the same machine; memory is the peak resident memory of the process. GPU: one RTX 5090, plain against fast (estimator batch and folded copy).

10 features, 20,000 query rows and one estimator, prediction takes 5.9 to 6.0 s with both implementations (best of three, two alternating runs of each), while fitting takes 5.3 to 5.4 s instead of 6.7 s and peak memory falls from 1.4 to 1.05 GB. On an RTX 5090 the estimator batch makes the two smallest tables 1.9 and 3.2 times faster and the others 7 to 27% faster.

10.1 Vulkan backend

PyTorch reaches NVIDIA GPUs through CUDA and AMD GPUs through ROCm, which covers Linux and a subset of consumer cards; Intel GPUs and most integrated GPUs are left out. The package therefore has a second GPU backend built on Vulkan, which current GPU drivers support on Linux and Windows. It computes the folded network of the previous section, with the same two calls (encode a context, predict query rows), through eight compute kernels written in WGSL and compiled to SPIR-V by wgpu when the backend starts, so no SDK or compiler is needed on the user's machine: a tiled matrix product with optional fused normalization, bias, GELU and residual; tiled attention with an online softmax for long key sequences; an attention kernel for the few summary and inducing tokens; row normalization; the cell features; rotary embeddings; the learned query scaling; and a strided copy. Kernels address tensors through strided views, so the same kernel reads a column of cells, a row of cells or the leading tokens of every row without copies. The column statistics and rank lookups (a sort and a binary search per column) stay on the host in PyTorch, and the context stays in GPU memory between fit and prediction. Everything runs in float32; sine, cosine and erf use their own range reduction and polynomial approximations (error below $5 \cdot 10^{-7}$), so results do not depend on the precision of a driver's transcendental functions. Drivers reset a GPU whose job runs too long (2 s on Windows; during development a 20,000-row encode submitted as one job triggered a ring reset of the Linux amdgpu driver), so heavy dispatches are cut into slices of workgroups submitted in jobs of about $5 \cdot 10^{10}$ floating-point operations.

Tests compare the backend with the PyTorch path on random networks with missing values, constant columns, padded features and several architecture variants: logits agree within about 10^{-6} and classifier probabilities within $3 \cdot 10^{-5}$. Table 9 times fit plus predict on an AMD RX 7900 XT: the backend is 6.0 to 10.3 times faster than the 16 threads of the host CPU and as fast as PyTorch with ROCm on the same GPU, or faster. One estimator's training cells must fit a single GPU buffer (2 GB, about 8.4 million cells); above that the classifier falls back to the CPU. Two general compilers were tried first: IREE had no attention lowering for Vulkan and generated incorrect code for RDNA3 on part of the network, and ONNX Runtime's WebGPU provider computed the fused attention with errors of 2 to 17%.

train rows × features, test rows	CPU, 16 threads	ROCm	Vulkan	Vulkan vs CPU
1,000 × 20, 500	0.45 s	0.057 s	0.044 s	10.3×
5,000 × 50, 2,000	4.84 s	0.59 s	0.50 s	9.6×
10,000 × 100, 5,000	19.8 s	2.48 s	2.13 s	9.3×
20,000 × 50, 5,000	29.2 s	4.76 s	4.86 s	6.0×
20,000 × 200, 10,000	70.4 s	10.4 s	8.68 s	8.1×

Table 9. Fit plus predict time of the B4 architecture with four estimators on synthetic binary tables: Intel i7-13700KF (16 threads), and an AMD RX 7900 XT through PyTorch 2.13 with ROCm 7.2 or through the Vulkan backend (Mesa RADV driver, Linux).

10.2 Software package

The package (`pip install lightpfn`) exposes the model as a scikit-learn classifier: `fit` validates the data and encodes the context, `predict_proba` runs the query rows, and the estimator works with pipelines, cloning and grid search. It accepts pandas data frames: columns of categorical, string, object or Boolean type become ordinal codes of the categories seen in training, with missing values and unseen categories as missing values. `device="auto"` picks a CUDA or ROCm GPU through PyTorch, then a Vulkan GPU, then the CPU. The weights are downloaded as safetensors with a JSON configuration from a Hugging Face commit pinned in the package; local checkpoints are read with PyTorch's restricted weights-only loader, and no code from a model repository is ever executed. The wheel holds only the inference code and depends on PyTorch, NumPy, scikit-learn, huggingface-hub and safetensors; the source repository also holds the priors, the trainer and the evaluation harness of this report.

11 TabArena

TabArena [7] is the benchmark that the selection protocol keeps aside. Version 0.1 has 51 tasks; we use its 38 classification tasks (30 binary, 8 multiclass). LightPFN has no regression head and none of the 13 regression tasks was evaluated. Only the two finalists of the final run are tested, F120 and its long-context continuation LC39k, after the choice between them had been made on D1, D3 and the scale test. This is the first look at TabArena for this series, and no decision in this report depends on it.

11.1 Evaluation with our harness

Protocol. Our harness (`lightpfn.eval.harness`) runs two protocols. The lite protocol reduces every task to at most 1,000 rows, stratified, and 100 random features, and evaluates it by stratified five-fold cross-validation (190 splits). The full protocol uses the official OpenML splits on the complete tables, from 748 to 150,000 rows and up to 1,776 features, with the first repeat only (three folds, 114 splits), where TabArena averages three or ten repeats. Above 20,000 training rows, which happens on 8 tasks, LightPFN subsamples its context with stratification. The baselines are CatBoost, LightGBM, XGBoost and random forest at default settings, run by the same harness, and TabICLv2 [22] (28M parameters, tabicl 2.2.0, default settings, on a GPU). These evaluations do not reproduce the official leaderboard protocol, which includes default, tuned and ensembled entries and ranks them by Elo: here seven fixed configurations are compared on the same splits. Our lite protocol is also distinct from official TabArena-Lite, which uses the first official split of each dataset. The error of a task is $1 - \text{AUC}$ for binary tasks and the log loss for multiclass tasks, as in TabArena; the AUC columns report the macro one-vs-rest AUC for every task. Models are ranked on each task by error and the ranks are averaged; the share of tasks on which a model beats default CatBoost counts ties as one half.

Results. Table 10 gives both protocols. On the lite protocol, LC39k with four estimators has similar aggregate scores to TabICLv2: mean AUC .8363 against .8376, mean rank 2.26 against 2.24, and the lower

	lite: ≤1,000 rows, 190 splits			full: official splits, 114		
Model	AUC	rank	> CatBoost	AUC	rank	> CatBoost
TabICLv2 (28M, GPU)	.8376	2.24	82%	.8637	1.58	87%
LightPFN LC39k, 4 estimators	.8363	2.26	84%	.8581	2.50	76%
LightPFN LC39k, 1 estimator	.8350	2.87	76%	.8568	3.37	68%
CatBoost, default	.8294	3.79		.8553	3.50	
LightGBM, default	.8104	5.37	18%	.8440	5.34	5%
XGBoost, default	.7966	6.24	8%	.8332	5.82	11%
Random forest, default	.8152	5.24	11%	.8383	5.89	3%
LightPFN F120, 4 / 1 estimators	.8360 / .8342	2.34 / 2.89	79% / 76%			

Table 10. TabArena, 38 classification tasks. Mean AUC over tasks, mean rank among the seven configurations above the line (error: 1−AUC for binary tasks, log loss for multiclass tasks), and share of tasks with lower error than default CatBoost. F120, the pretrained model before the long-context stage, was evaluated on the lite protocol only; its ranks are computed with F120 in place of LC39k.

error on 47% of the tasks. It beats default CatBoost on 84% of the tasks, LightGBM on 89%, random forest on 87% and XGBoost on 95%. F120 is close behind (AUC .8360, beats CatBoost on 79%), with no apparent degradation on tables of at most 1,000 rows. On the full protocol, LC39k with four estimators is second among the seven configurations, with mean AUC .8581 and rank 2.50. TabICLv2, six times larger and run on a GPU, leads clearly (.8637, rank 1.58) and has the lower error on 82% of the tasks. LC39k beats default CatBoost (.8553, rank 3.50) on 76% of the tasks, LightGBM on 89%, random forest on 92% and XGBoost on 87%. With one estimator it keeps rank 3.37, ahead of CatBoost, and beats it on 68% of the tasks. The four-estimator mean AUC difference from CatBoost is +0.28 points with a paired 95% bootstrap interval of $[-0.26,\ 0.80]$; the number of wins and mean rank therefore do not establish a positive mean AUC difference. Against TabICLv2 the difference is −0.56 points $[-0.98,\ -0.25]$. These results do not determine a position on the official leaderboard.

Table 11 splits the full protocol by task type. The lead over CatBoost is largest on small tables (fewer than 2,500 rows: all 12 tasks, +1.18 AUC points) and on multiclass tasks (all 8 tasks in log loss). On tables of at least 10,000 rows LightPFN still has the lower error on 69% of the tasks, but its mean AUC is 0.40 points below CatBoost's, pulled down by a few datasets; on tasks whose minority class holds 1 to 10% of the rows it beats CatBoost on 64% of them and is 0.35 points below in mean AUC. With one estimator these two groups drop to 50% and 45% of the tasks. Figure 9 shows the tasks one by one. LC39k has the higher AUC on 28 of 38 and loses most on *in_vehicle_coupon_recommendation* (−4.49 points), *Amazon_employee_access* (−3.95), *Diabetes130US* (−2.35), *kddcup09_appetency* (−1.96), *polish_companies_bankruptcy* (−1.72) and *Bioresponse* (−1.61). The first three are mostly categorical (88, 100 and 83% of the columns), and LightPFN reads category codes as numbers, while CatBoost encodes categories with ordered target statistics; TabICLv2, which also reads codes as numbers, trails CatBoost on *Amazon_employee_access* too (−3.49). On *hiva_agnostic* every model is close to chance (AUC .47 to .52).

Cost. Table 12 and Figure 10 put the ranks next to the time per split. The timings come from two hosts and four kinds of processor, so they are indicative only. The tree ensembles ran on the CPU of a desktop with an Intel i7-13700KF, five jobs of four threads at a time, and TabICLv2 on its AMD Radeon RX 7900 XT. LightPFN ran on the rented server, on its EPYC 9654 CPU with eight jobs of 12 threads at a time (six of eight threads on the lite protocol) and on one RTX 5090. In these measurements LightPFN has the highest median CPU time on medium and large tables, although not on every task (on *splice* it takes 5.6 s per split against 34 s for CatBoost). With four estimators and the fast path it takes a median of 15 s per split on medium tables and 54 s on large ones, against 4 and 7 s for CatBoost and less than half a second for LightGBM, XGBoost and random forest; on small tables it takes 1.0 s, less than CatBoost's 2.9 s. One estimator costs about a quarter as

		mean rank				LC39k ×4 vs CatBoost	
group	tasks	LC39k ×4	LC39k ×1	CatBoost	TabICLv2	AUC points	tasks won
all	38	2.50	3.37	3.50	1.58	+0.28	76%
binary	30	2.60	3.67	3.30	1.57	+0.25	70%
multiclass	8	2.13	2.25	4.25	1.63	+0.38	100%
small, <2,500 rows	12	2.25	2.33	4.25	1.42	+1.18	100%
medium, 2,500 to <10,000	10	2.70	3.70	3.20	1.80	+0.28	60%
large, ≥10,000	16	2.56	3.94	3.13	1.56	−0.40	69%
balanced, minority ≥10%	25	2.52	3.40	3.68	1.44	+0.51	80%
imbalanced, 1 to <10%	11	2.55	3.55	2.91	1.82	−0.35	64%
extreme, <1%	2	2.00	2.00	4.50	2.00	+0.86	100%

Table 11. TabArena full protocol by task group: mean rank among the seven configurations of Table 10, difference of mean AUC to default CatBoost, and share of tasks with lower error than CatBoost, for LC39k with four estimators. Size groups count all rows of the dataset.

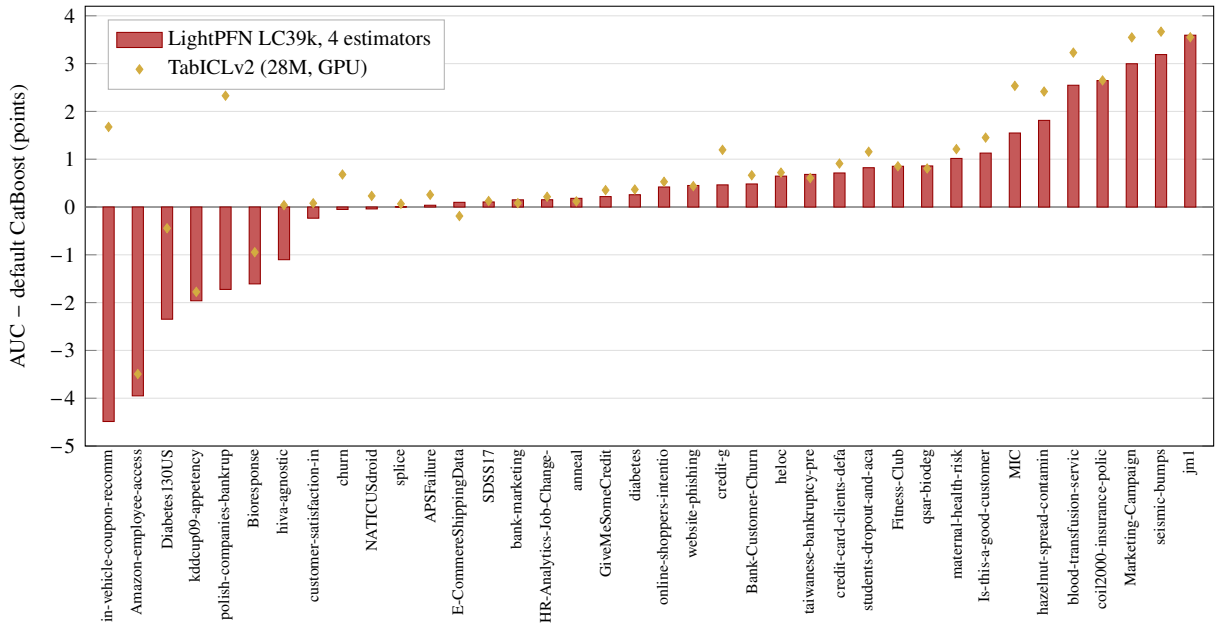


Figure 9. TabArena full protocol: AUC difference to default CatBoost on each of the 38 classification tasks, in points, for LightPFN LC39k with four estimators (bars, sorted) and TabICLv2 (diamonds). Ranks use the log loss for the 8 multiclass tasks, so the share of tasks won in Table 10 (76%) differs slightly from the 28 of 38 tasks with higher AUC here.

much (3.9 and 13.5 s) and keeps a mean rank ahead of CatBoost. Prediction alone takes 4.1 s per 1,000 rows on average with four estimators, against 0.02 s for CatBoost. On the GPU, four estimators take a median of 0.21 s per split and 1.2 s on large tables; TabICLv2 takes 1.5 s on a different GPU.

The rerun with the new defaults (Section 10) changes the predictions only by floating-point rounding. On the CPU, against the initial evaluations, the AUC of a split moves by at most $3.5 \cdot 10^{-5}$, the log loss by at most $2.1 \cdot 10^{-7}$ and the accuracy by at most $2 \cdot 10^{-5}$ (one of the 50,000 test rows of *GiveMeSomeCredit* changes class). On the GPU, against the earlier implementation on the same GPU, the log loss moves by at most $2.2 \cdot 10^{-7}$ and the AUC by up to $3.4 \cdot 10^{-4}$; these larger AUC movements come from tied or nearly tied probabilities that change order, for instance among the 588 duplicated rows of *heloc*. Against the initial CPU evaluations the GPU rerun differs by up to $4.4 \cdot 10^{-4}$ in AUC and $2.3 \cdot 10^{-6}$ in log loss. Every rank is

			median fit + predict per split (s)				predict
Model	hardware	rank	all	small	medium	large	s / 1k rows
LightGBM, default	i7, 4 threads	5.34	0.23	0.15	0.32	0.29	0.004
XGBoost, default	i7, 4 threads	5.82	0.13	0.05	0.16	0.48	0.015
Random forest, default	i7, 4 threads	5.89	0.22	0.15	0.21	0.48	0.024
CatBoost, default	i7, 4 threads	3.50	4.94	2.87	4.25	6.71	0.018
TabICLv2	RX 7900 XT	1.58	1.50	0.49	3.20	5.45	2.87
LightPFN ×4, plain	EPYC, 12 threads	2.50	16.25	1.09	40.23	64.55	10.26
LightPFN ×4, fast	EPYC, 12 threads	2.50	11.58	1.03	15.43	54.06	4.12
LightPFN ×4, fast	RTX 5090	2.50	0.21	0.02	0.24	1.18	0.06
LightPFN ×1, plain	EPYC, 12 threads	3.37	4.59	0.27	10.23	16.57	2.57
LightPFN ×1, fast	EPYC, 12 threads	3.37	2.95	0.23	3.87	13.45	1.05
LightPFN ×1, fast	RTX 5090	3.37	0.07	0.02	0.07	0.34	0.02

Table 12. TabArena full protocol: mean rank and cost. Median over tasks of the mean fit plus predict time of a split (12 small, 10 medium and 16 large tasks), and mean prediction time per 1,000 test rows. Plain: the initial runs; fast: the rerun with the new defaults, same predictions up to rounding and same ranks. The i7 runs had five concurrent jobs and the EPYC runs eight; the hosts differ, so the comparison across rows is indicative.

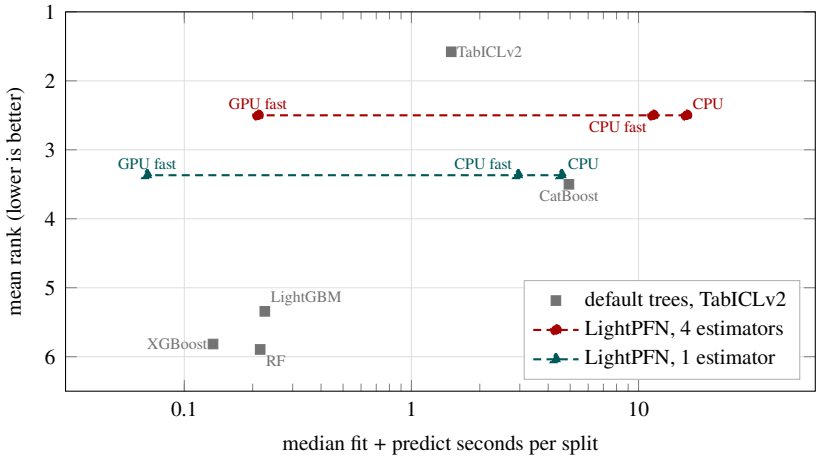


Figure 10. TabArena full protocol: mean rank against the median fit plus predict time per split (log scale). Each LightPFN line joins the initial CPU evaluation, the CPU rerun with the new inference defaults and the GPU rerun, which have the same predictions up to rounding and the same rank. Hardware as in Table 12; the different hosts and concurrency settings prevent a controlled comparison across models.

identical on both protocols. The summed CPU time of full-protocol splits falls from 11,792 to 5,209 s with four estimators (2.26 times) and from 3,046 to 1,317 s with one (2.31 times). The median per-task gain is 1.20 times, reaching 2.8–3.3 on wide tables. Prediction on the three longest narrow tables is 21–28% slower in the concurrent rerun; the idle-machine control in Section 10 finds similar prediction times, consistent with contention. Lite-protocol CPU gains are 1.14 and 1.18 times for four and one estimators. On the GPU, four estimators gain 1.15 times on full tables and 2.3 times on lite tables; one estimator shows no gain on full tables and about 4 ms additional overhead per lite split.

11.2 Official TabArena leaderboard

Separately from the harness above, the released model was evaluated through TabArena's own pipeline: the default LightPFN configuration with four estimators, one checkpoint and no HPO search space, with the arena's eight-fold inner bagging and `8x1` validation protocol. The wrapper keeps encoded contexts only in

Position	Method	Elo	95% interval offsets
1	LimiX-2 (default)	1927	+137/−77
2	Kumo-Tabular (default)	1918	+163/−112
14	TabICLv2 (default)	1575	+72/−69
18	TabDPT-1.3 (default)	1524	+75/−57
19	RealMLP (tuned + ensembled)	1474	+53/−43
20	TabM (tuned + ensembled)	1452	+56/−40
21	LightPFN (default)	1441	+69/−79
22	LightGBM (tuned + ensembled)	1426	+43/−29
24	CatBoost (tuned + ensembled)	1410	+53/−44
27	CatBoost (tuned)	1400	+47/−43
31	CatBoost (default)	1387	+44/−45
35	XGBoost (tuned + ensembled)	1373	+40/−46
60	XGBoost (default)	1230	+39/−46
66	LightGBM (default)	1191	+50/−41
87	RandomForest (default)	1000	+62/−65

Table 13. Official TabArena leaderboard, classification, all splits (11 October 2026): selected entries among 92 methods, with 95% Elo interval offsets. Four LightPFN estimators, run and verified by the maintainers, none imputed. Separate from our harness in Table 10.

the refit model; fold models build and release them at prediction. After the model class was merged into TabArena, the maintainers ran it on all splits of the 38 classification datasets and marked the result as verified, with no failed or imputed LightPFN result. No model or recipe was selected on these results.

On the leaderboard of 11 October 2026[1] LightPFN is 21st of 92 entries by Elo, with Elo 1441 and a 95% interval of [1362, 1510] (+69/−79); on the Lite view, which uses only the first split of each dataset, it is 20th of 92 with Elo 1426 (+71/−70). Table 13 lists selected entries. LightPFN's point estimate exceeds every tree-based entry, including tuned and ensembled ones, but its interval overlaps those of default CatBoost (1387, [1342, 1431]) and of the tuned gradient-boosting entries; these marginal intervals do not establish a significant pairwise advantage. Default XGBoost and LightGBM are 211 and 250 points lower, with separate intervals. In TabArena's win-rate matrix LightPFN beats tuned and ensembled CatBoost, LightGBM and XGBoost in 58%, 58% and 64% of comparisons. The first 18 places are all foundation models; TabICLv2 leads by 134 points and wins 75% of its comparisons with LightPFN. By subset, LightPFN is 19th of 92 on the 28 small datasets, above every tree-based entry, and 44th of 93 on the 10 medium datasets, where default CatBoost is 17th, consistent with the larger-table results of Section 9. The default view of the leaderboard also includes the 13 regression datasets, which LightPFN does not support; there its results are imputed and it is 42nd of 99.

TabArena's timings put LightPFN at a median of 0.26 s to train and 0.029 s to predict per 1,000 rows, on a GPU. Figure 11 plots Elo against training time: LightPFN is on the Pareto front, and only KNN trains faster. Against prediction time it is also on the front, with only EBM and default CatBoost faster. Tree-based methods were timed on a CPU, so the time axis compares the setups as run, not algorithms on equal hardware.

Before the maintainers' run we evaluated the Lite subset ourselves on 8 October 2026. That run used `TabArenaV0pt1ExperimentBundle` with the `lite, classification` subset, on an Intel i7-13700KF, 32 GB RAM and an AMD RX 7900 XT (Linux, ROCm 7.2, PyTorch 2.13.0, Python 3.12.15). All 38 tasks succeeded; against the leaderboard of that day LightPFN was 25th of 99 with Elo 1420 (+67/−66), with median fit and prediction times of 0.65 s and 0.086 s per 1,000 rows. The entry-point script imports Torch before AutoGluon so its GPU discovery uses ROCm rather than NVIDIA-only NVML. The results archive

[1] https://huggingface.co/spaces/TabArena/leaderboard, data commit `0aac376`; view: Classification, imputed models excluded.

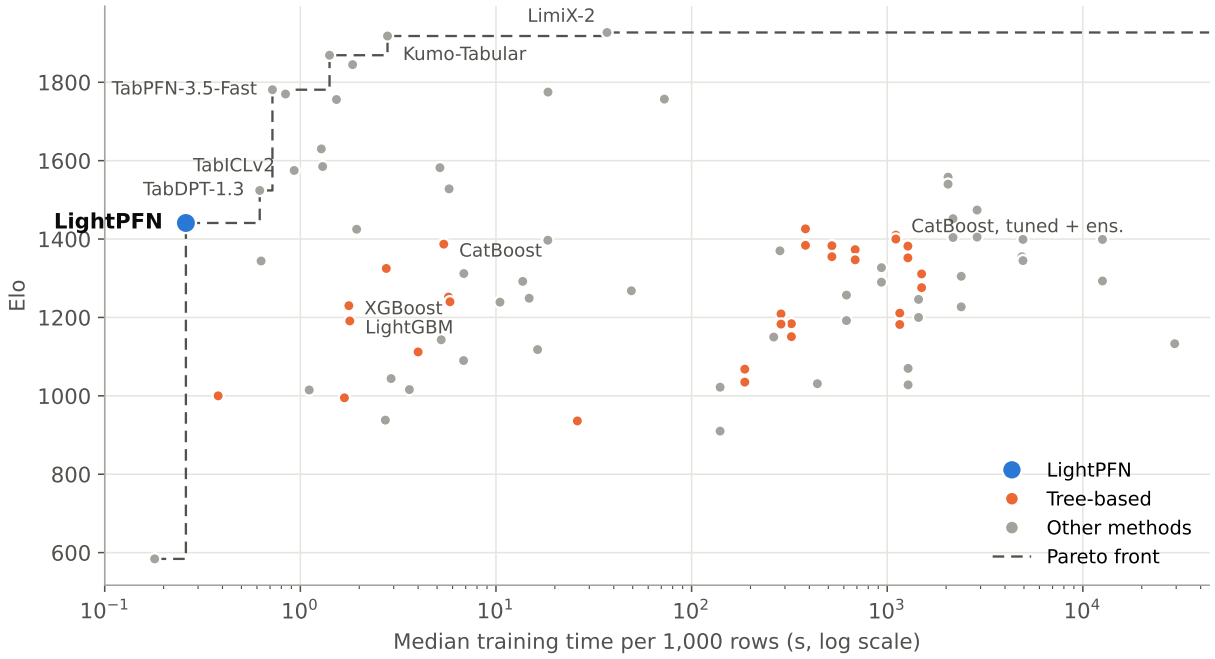


Figure 11. Official TabArena leaderboard, classification, all splits: Elo against median training time per 1,000 rows (log scale) for 92 methods, with the Pareto front. LightPFN in blue, tree-based methods in orange. Timings are TabArena's: GPU methods on a GPU, tree-based methods on a CPU; tuned entries include their search time.

includes the script, 38 raw result files and the evaluation CSVs.[2]

12 Categorical features

LightPFN reads categorical columns as ordinal codes. Its largest TabArena losses are on mostly categorical tables (Section 11), and on D4 it trails default CatBoost on the datasets with categorical columns of hundreds to thousands of levels (Section 9). Removing the categorical columns with more than 50 levels at 10,000 training rows measures what each model draws from them (Table 14): CatBoost loses 0.4 to 2.0 AUC points, 1.0 on average, and LightPFN 0.2 on average. On one synthetic lookup column the model keeps pace with CatBoost up to 500 categories (AUC 0.881 against 0.885) and falls behind at 2,000 (0.864 against 0.892). Recognizing a category is therefore not the difficulty; using many rare categories, each with a small effect of its own, is.

We tested three remedies that keep the pretrained weights. Selection used D3 and Dcat, the twelve cells formed by the six D4 datasets with categorical columns of more than two levels at 10,000 and 25,000 training rows; Dcat extends the protocol of Section 6 outside TabArena.

1. Thirteen inference-time encodings on D3: frequency encoding, one-hot encoding up to 16 levels, random category codes, smoothed target encoding and leave-one-out statistics, used alone or alongside the ordinal codes. None yields a mean AUC gain with an interval above zero. Target encodings lose 0.3 to 0.5 points. Leave-one-out statistics tie each context row's feature to its own label and cause a collapse of about 12 AUC points.

2. The count columns of Kumo Tabular [18], one $\log(1 + \text{count})$ column per categorical column with more than 50 levels: +0.03 points $[-0.19,\ 0.23]$ on Dcat.

[2] https://github.com/GioOtto/LightPFN/releases/tag/v1.0.0.

dataset	columns >50 levels (largest)	LightPFN gain	CatBoost gain
airlines	2 (266)	+0.54	+1.02
albert	36 (6,006)	+0.17	+1.12
kick	5 (711)	+0.07	+0.36
okcupid-stem	3 (1,951)	+0.29	+0.92
sf-police-incidents	1 (5,206)	+0.49	+2.02
porto-seguro	1 (104)	−0.28	+0.42

Table 14. AUC points each model gains from the categorical columns with more than 50 levels: AUC with all columns minus AUC without those columns, D4 datasets at 10,000 training rows, LightPFN with one estimator and the whole training set as context, CatBoost at default settings.

3. A categorical adapter of 6,384 parameters: separate Fourier frequencies and projection for categorical cells, as in Kumo Tabular, and the ordered target statistics of CatBoost [19] for every categorical cell, embedded through the label embedding of the column stage, plus count features. A context row uses only the context rows that precede it in a random order, so its own label never enters its statistics; query rows use all context rows. The adapter starts as the identity, and tables without categorical columns get the released predictions bit for bit. Fine-tuned with all other weights frozen, it does not improve Dcat: -0.06 points $[-0.22,\ 0.07]$ after 4,000 steps on pretraining-size tables of graph v4 and rule tasks, and after 1,000 steps on a mix in which 60% of the tasks have 1,024 to 10,240 rows, +0.01 $[-0.06,\ 0.08]$ for the moving average and -0.20 $[-0.38,\ -0.04]$ for the raw weights.

The mechanism works: gradients reach the adapter, the statistics match a direct computation and leak no label. The limit is the prior. A high-cardinality column of graph v4 is a quantized numeric variable with permuted codes, which the frozen network already exploits, so per-category statistics do not lower the training loss; the real columns where CatBoost gains have many rare levels with small effects of their own, and no prior task resembles them. Version 2 will add native categorical handling at pretraining time: the categorical mask as a model input from the start and a prior family of categorical random effects. The adapter stays in the code, off by default.

13 Discussion and limitations

Interactions. The rule prior and the complete B4 configuration are complementary on the mechanism probes. This supports preserving feature interactions before compression, but neither isolates the row and column refinement effects nor measures how frequent pure interactions are in real datasets.

Long contexts. Further training improves larger-table performance without increasing the parameter count. Most gains arrive early, including before the longest tables enter training; the experiment therefore does not isolate context length from additional optimization. Remaining losses concentrate on a few datasets, and inference retains a quadratic context-attention term. Training beyond 60,000 rows and inference in the 300,000-row regime remain untested.

Categorical inputs. Several of the largest TabArena losses occur on mostly categorical tables, and on large tables CatBoost draws much more from high-cardinality columns. Section 12 shows that neither encodings nor an adapter on the frozen network recover that signal; the evidence points at the prior, which has to be tested by pretraining with categorical random effects.

Cost. A small parameter count does not remove the cost of attention over long contexts. Cache blocking improves the cell stages, while context size and estimator count still govern substantial work. The reported cross-host timings indicate a cost concern on larger tables; controlled measurements on an idle laptop CPU are needed before claiming comparable latency to boosted trees. Reducing context or estimator count would

also change predictions and requires a separate accuracy evaluation.

Limitations. All ablations use one training seed and 8,000 steps; the hardware anchor illustrates a D3 shift of 0.3 points without a recipe change, rather than estimating run-to-run variance. D3 follows a lite protocol with at most 1,000 rows and 100 features. The scale test covers larger tables with only 15 datasets, six of them at 16,000 and 32,000 rows, and it also guided the long-context stage; D4 has 13 datasets and one repeat, and its intervals are wide. Baselines on D3 and in the scale test use default hyperparameters and a single model, as do our TabArena harness runs. The official leaderboard comparison includes tuned and ensembled entries and all splits, but its Elo intervals are marginal, not pairwise tests. The final run is a single run, and its comparison with C2 mixes longer training with the larger batch and fresh data. Both TabArena evaluations cover the 38 classification tasks only; our harness uses the first of three or ten repeats. On D3 the XGBoost wrapper fails on one task, so its predictive comparison excludes that task from both models. Class counts above ten, exact feature-order invariance, and the isolated effects of row and column refinement remain unverified.

14 Conclusion

LightPFN combines summary-mediated row refinement, a second column stage and a rule prior within a 4.60-million-parameter budget, using only synthetic training data. The final long-context model reaches 0.911 mean AUC on D3, 0.86 points above default CatBoost, and improves performance on larger tables without a clear change on the small-table sets. On the scale test and D4, its mean differences from default CatBoost have intervals that include zero at the larger training sizes; this does not establish equivalence, and several datasets remain 1 to 4 AUC points behind.

On the 38 TabArena classification tasks and one repeat of the official splits, four estimators rank second among seven evaluated configurations, behind TabICLv2, and have lower error than default CatBoost on 76% of tasks. This harness comparison uses fixed defaults. On the official TabArena leaderboard, run and verified by the maintainers on all splits, LightPFN is 21st of 92 entries with Elo 1441 ($+69/-79$), above the point estimates of every tree-based entry, with intervals overlapping default CatBoost and the tuned gradient-boosting entries, and on the Pareto front of Elo against training and prediction time. Cache blocking and folded normalization improve CPU inference time by up to 4.1 times without changing predictions beyond floating-point rounding, and a Vulkan backend runs the same network on GPUs of any vendor. Competitive latency against boosted trees on an idle laptop CPU remains to be measured. Code, weights and the training pipeline are released under Apache 2.0 with a scikit-learn package. Version 2 will add native categorical features, pretrained with a prior of high-cardinality categorical columns, and regression, within the same parameter budget.

Declaration of generative AI and AI-assisted technologies

During the preparation of this work the author used generative AI tools as coding and writing assistants: to write and review parts of the code, to run and analyse experiments, and to draft and edit the text of this report. The author reviewed and edited the results and the text as needed and takes full responsibility for the content of this report.

A Hyperparameters

Cell embedding	dimension 64; offsets $\{0, 1, 3\}$; 16 learned frequencies per offset; 4 rank frequencies
Column encoding	ISAB, 2 layers, 4 heads, 64 inducing tokens
Row refinement	$R = 3$, $K = 4$ temporary tokens; 4 broadcasts, 3 gathers, 4 heads
Column refinement	independent second ISAB stage, same dimensions as column encoding
Row compression	4 separate readout tokens, 3 blocks, 4 heads; rotary positions on cell keys
ICL	7 blocks (8 in A0), dimension 256, 8 heads, 1 key/value head for queries, 16 thinking rows
Decoder	retrieval attention, 4 heads, 16 label slots
MLP ratio	2 everywhere
Optimizers	Muon for block matrices, AdamW (0.9, 0.98) elsewhere; weight decay 0.01
Learning rate	10^{-3}, linear warmup, cosine to 10^{-4}
Ablations	8,000 steps × 32 tasks, warmup 800
Final run	120,000 steps × 64 tasks, warmup 2,000, two GPUs
Long-context stage	from the final checkpoint; learning rate 10^{-4}, warmup 200, cosine to 10^{-5}; 8 groups per GPU from pools of 256–2,048, 1,024–10,240 and 4,096–60,000 rows (15/35/50% of the groups); micro-batch cost $n(m + 8) \leq 1.1 \times 10^6$ with m the real features, rows subsampled above it; 39,250 steps
Other	clip 1.0; EMA 0.999; bfloat16 autocast; 8×10^5 cells per micro-batch (final run)
Missingness	30% of tasks; MCAR, MAR, MNAR; rate LogU(0.01, 0.5) per column

Table 15. Model and training hyperparameters. Series A and B used 4×10^5 cells per micro-batch and series C 10^6; the budget changes memory use while preserving the task-weighted objective, up to floating-point accumulation effects.

References

[1] Y. Benjamini and Y. Hochberg. Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society, Series B*, 57(1):289–300, 1995.

[2] B. Bischl, G. Casalicchio, M. Feurer, P. Gijsbers, F. Hutter, M. Lang, R. G. Mantovani, J. N. van Rijn, and J. Vanschoren. OpenML benchmarking suites. In *NeurIPS Datasets and Benchmarks Track*, 2021.

[3] L. Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[4] T. Chen and C. Guestrin. XGBoost: a scalable tree boosting system. In *KDD*, 2016.

[5] M. Cho, M. Jeong, D. Lee, J. Lee, and J. Yoo. Causilo technical report. arXiv:2609.22866v1, 2026. https://arxiv.org/abs/2609.22866v1.

[6] T. Darcet, M. Oquab, J. Mairal, and P. Bojanowski. Vision transformers need registers. In *ICLR*, 2024.

[7] N. Erickson, L. Purucker, A. Tschalzev, D. Holzmüller, P. M. Desai, D. Salinas, and F. Hutter. TabArena: a living benchmark for machine learning on tabular data. 2025.

[8] P. Geurts, D. Ernst, and L. Wehenkel. Extremely randomized trees. *Machine Learning*, 63(1):3–42, 2006.

[9] N. Hollmann, S. Müller, K. Eggensperger, and F. Hutter. TabPFN: a transformer that solves small tabular classification problems in a second. In *ICLR*, 2023.

[10] N. Hollmann, S. Müller, L. Purucker, A. Krishnakumar, M. Körfer, S. B. Hoo, R. T. Schirrmeister, and F. Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637:319–326, 2025.

[11] K. Jordan, Y. Jin, V. Boza, J. You, F. Cesista, L. Newhouse, and J. Bernstein. Muon: an optimizer for hidden layers in neural networks. Technical blog post, 2024.

[12] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu. LightGBM: a highly efficient gradient boosting decision tree. In *NeurIPS*, 2017.

[13] J. Lee, Y. Lee, J. Kim, A. Kosiorek, S. Choi, and Y. W. Teh. Set Transformer: a framework for attention-based permutation-invariant neural networks. In *ICML*, 2019.

[14] J. Liu et al. Muon is scalable for LLM training. arXiv preprint, 2025.

[15] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.

[16] S. Müller, N. Hollmann, S. Pineda Arango, J. Grabocka, and F. Hutter. Transformers can do Bayesian inference. In *ICLR*, 2022.

[17] T. Nagler. Statistical foundations of prior-data fitted networks. In *ICML*, PMLR 202:25660–25676, 2023. https://proceedings.mlr.press/v202/nagler23a.html.

[18] NVIDIA. Kumo Tabular, reference implementation in structured-data-models, 2026. https://github.com/NVIDIA/structured-data-models.

[19] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin. CatBoost: unbiased boosting with categorical features. In *NeurIPS*, 2018.

[20] Prior Labs. TabPFN-3: model release and technical documentation, 2026.

[21] J. Qu, D. Holzmüller, G. Varoquaux, and M. Le Morvan. TabICL: a tabular foundation model for in-context learning on large data. In *ICML*, 2025.

[22] TabICLv2: technical report and reference implementation (tabicl 2.2.0), 2026.

[23] N. Shazeer. Fast transformer decoding: one write-head is all you need. arXiv preprint, 2019.

[24] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. RoFormer: enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

[25] M. Tancik, P. P. Srinivasan, B. Mildenhall, S. Fridovich-Keil, N. Raghavan, U. Singhal, R. Ramamoorthi, J. T. Barron, and R. Ng. Fourier features let networks learn high frequency functions in low dimensional domains. In *NeurIPS*, 2020.

[26] X. Zhang et al. LimiX: unleashing structured-data modeling capability for generalist intelligence. arXiv preprint, 2025.

[27] X. Zhang et al. Mitra: mixed synthetic priors for enhancing tabular foundation models. 2025.